

RFC

Network Working Group  
Request for Comments: 0001  
Category: Experimental

CSNETWK  
De La Salle University - Manila  
A. F. B. Laguna  
April 2026

# Magic: The Gathering Multiplayer Network Protocol

Version 1.0 (MTGNP)

---

## Abstract

This document specifies the Magic: The Gathering Multiplayer Network Protocol (MTGNP), version 1.0. MTGNP defines a TCP-based, message-oriented, client-server protocol for conducting two-player, simplified Magic: The Gathering card game sessions over a network. The protocol addresses game state synchronization, turn management, priority arbitration, stack resolution, and combat mechanics. It is intended as an educational reference for implementing networked card game systems.

## 1. Introduction

Magic: The Gathering (MTG) is a complex collectible card game with intricate rules governing simultaneous player decisions, ordered action queues (the stack), and hidden game state. Implementing MTG over a network presents unique protocol challenges not found in simpler multiplayer games: priority windows allow both players to act at nearly every point in a game turn, and game state must be kept synchronized across clients while preserving hidden information such as each player's hand.

MTGNP defines how a central server (the Game Server) mediates between two clients. The server is the sole source of truth for all game state and validates every player action. Clients are intentionally thin: they render state received from the server and transmit player actions, but they never compute authoritative game outcomes.

This document specifies a simplified subset of the full MTG rules. Specifically, the following limitations apply to MTGNP 1.0:

- Exactly two players per game.
- Decks of between 1 and 50 cards each, drawn from a fixed, pre-defined card set. Both players may use different deck sizes.
- No replacement effects.
- No planeswalker permanents.
- No match structure (no best-of-three). After `GAME_OVER`, both players may immediately start a new game on the same TCP connection by sending fresh `PLAYER_READY` PDUs.

Future revisions may relax these limitations.

**NOTE:** MTGNP 1.0 does not define a card data transfer mechanism. Card costs, effects, power, toughness, and ability text are assumed to be pre-loaded by both the server and all clients from a shared out-of-band card catalog (e.g., a static JSON file distributed with the implementation). The card IDs exchanged in PDUs are keys into this shared catalog.

## 2. Requirements Language

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in RFC 2119 [RFC2119].

## 3. Terminology

The following terms are used throughout this document:

**Active Player (AP):** The player whose turn it currently is.

**Non-Active Player (NAP):** The player who is not currently taking their turn.

**Priority:** The right to take a game action. Only the player who holds priority may cast spells or activate non-mana abilities.

**The Stack:** A last-in, first-out (LIFO) zone where spells and abilities wait before resolving. Both players may add items to the stack whenever they hold priority.

**Sequence Number (seq\_num):** A monotonically increasing integer present in every PDU. For server-to-client PDUs, the server increments this counter with each PDU it sends. For client-to-server action PDUs, the client MUST echo the seq\_num from the most recently received PRIORITY\_GRANT or corresponding server request PDU; the server MUST reject mismatches with ERROR code STALE\_ACTION.

**Game State:** The complete authoritative set of all game information: all zones (library, hand, battlefield, graveyard, stack), life totals, turn number, and current phase.

**Visible State:** The subset of Game State visible to a specific player. Each player's hand is hidden from the opponent; all other zones are public.

**Phase:** A major division of a turn (e.g., Main Phase, Combat Phase).

**Step:** A subdivision of a phase (e.g., Declare Attackers Step, Declare Blockers Step).

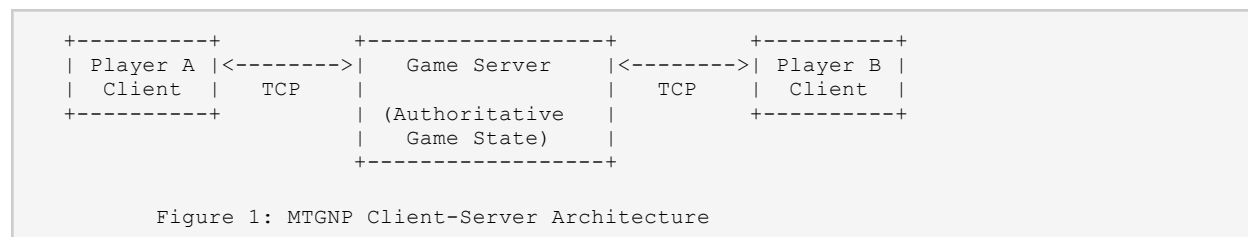
**Summoning Sickness:** A creature that entered the battlefield under a player's control this turn MUST NOT be declared as an attacker and MUST NOT activate abilities with the tap symbol in their cost, unless the creature has Haste. The server enforces this rule automatically.

**PDU:** Protocol Data Unit. A single MTGNP message exchanged between client and server.

## 4. System Architecture

### 4.1. Client-Server Model

MTGNP uses a centralized client-server model. One process acts as the Game Server; exactly two processes act as Player Clients.



### 4.2. Server Responsibilities

The Game Server MUST:

- Maintain the single authoritative copy of the Game State.
- Validate all PDUs received from clients and reject illegal actions with an ERROR message.
- Manage all phase and step transitions and broadcast PHASE\_TRANSITION messages.
- Issue PRIORITY\_GRANT messages to the appropriate player at the start of each priority window.
- Manage the Stack, resolving the top item when both players pass priority consecutively.
- Compute and apply all combat damage.
- Detect win/loss conditions and issue GAME\_OVER messages.
- Enforce the time\_limit\_ms advertised in each PRIORITY\_GRANT. If the priority-holding client does not respond before the deadline, the server MUST broadcast GAME\_OVER with reason DISCONNECT, retain the TCP connection for the non-timed-out player, and return to LOBBY state.
- Send personalized GAME\_STATE\_UPDATE messages to each client, filtering out hidden information.

### 4.3. Client Responsibilities

A Player Client MUST:

- Maintain a local rendering of the Visible State for its player.
- Include the current seq\_num in all action PDUs.
- Accept GAME\_STATE\_UPDATE messages from the server as the authoritative state and discard any locally computed state that conflicts.
- Send PING messages at regular intervals (RECOMMENDED: every 30 seconds) and disconnect if no PONG is received within an implementation-defined timeout (RECOMMENDED: 10 seconds after sending a PING with no response).

**NOTE:** Clients MUST NOT compute game outcomes locally. All game logic resides on the server. A client that attempts to validate actions locally risks displaying inconsistent state.

## 5. Transport and Message Framing

### 5.1. TCP Connection

MTGNP operates over TCP [RFC9293]. The default server port is 4444. Clients **MUST** initiate the TCP connection to the server. The server **MUST** accept connections from exactly two clients before beginning the game setup sequence. Additional connection attempts after two players are seated **MUST** be refused. Because TCP guarantees in-order delivery, MTGNP does not define any PDU reordering or deduplication mechanism beyond the seq\_num field.

### 5.2. Message Framing

All PDUs are framed with a 4-byte, big-endian unsigned integer length prefix indicating the byte length of the JSON payload that follows. Receivers **MUST** read exactly that many bytes before attempting JSON parsing. A PDU **MUST NOT** exceed 65,535 bytes.

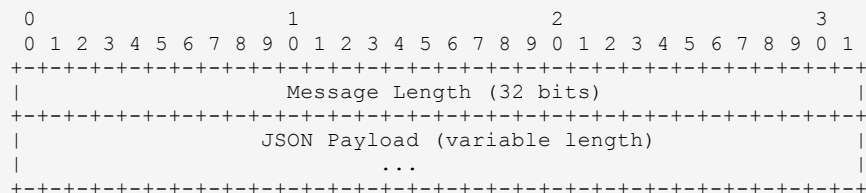


Figure 2: MTGNP Message Frame

### 5.3. Message Encoding

All PDUs are encoded as JSON objects [RFC8259]. All JSON **MUST** be valid UTF-8. Field names are case-sensitive and **MUST** use the exact names specified in Section 10 of this document.

### 5.4. General Message Structure

Every MTGNP PDU is a JSON object. Two fields **MUST** appear in every PDU, regardless of message type:

```
{
  "type":    "<MESSAGE_TYPE>", // REQUIRED: string identifier for this PDU
  "seq_num": <integer>,        // REQUIRED in every PDU
  // ... additional message-specific fields ...
}
```

type: A string literal that identifies the PDU kind. The receiver **MUST** inspect this field first to determine how to parse the remaining fields. The complete enumeration of valid type values is given in Section 10.

seq\_num: A monotonically increasing integer. For priority-bearing client-to-server PDUs (CAST\_SPELL, ACTIVATE\_ABILITY, PRIORITY\_PASS, DECLARE\_ATTACKERS, DECLARE\_BLOCKERS, ASSIGN\_DAMAGE\_ORDER, PLAY\_LAND, MULLIGAN\_CHOICE, DISCARD, TRIGGER\_ORDER\_RESPONSE, TRIGGER\_CHOICE\_RESPONSE), seq\_num MUST equal the value from the most recently received PRIORITY\_GRANT or the corresponding server request PDU. For MULLIGAN\_CHOICE specifically, the corresponding server request PDU is the GAME\_STATE\_UPDATE sent by the server at the start of the MULLIGAN phase (or after a redraw); the client MUST echo that PDU's seq\_num. For DISCARD specifically, the corresponding server request PDU is the GAME\_STATE\_UPDATE the server sends at Cleanup when the hand size exceeds seven; the client MUST echo that PDU's seq\_num. For DECLARE\_ATTACKERS, DECLARE\_BLOCKERS, and ASSIGN\_DAMAGE\_ORDER, the corresponding server request PDU is the PHASE\_TRANSITION that signals each respective step; the client MUST echo that PHASE\_TRANSITION's seq\_num. The server MUST reject any such PDU whose seq\_num does not match the current priority token and MUST respond with ERROR code STALE\_ACTION. For server-issued PDUs, seq\_num is the server's own monotonically increasing counter; receivers MAY use it for message ordering and duplicate detection. A simple counter that increments with each PDU sent is sufficient — seq\_num is not required to be globally unique across the full game session.

Two client PDUs are exempt from the priority-echo rule. CONCEDE MAY be sent at any time regardless of which player holds priority; its seq\_num MUST be the value from the most recently received server PDU of any type (not necessarily a PRIORITY\_GRANT). PING is a heartbeat PDU whose seq\_num is a client-maintained counter independent of the priority token; the server echoes it unchanged in PONG for round-trip correlation and does not validate it against the current priority token.

```
-- Player 1 acts on a stale priority grant (seq_num=14, current is 16) --  
  
C->S    { "type": "CAST_SPELL", "seq_num": 14,  
          "card_id": "counterspell_001", "targets": ["stk_02"],  
          "mana_payment": { "U": 2 } }  
  
S->P1   { "type": "ERROR", "seq_num": 15, "code": "STALE_ACTION",  
          "message": "Priority token mismatch. Expected seq_num 16, got 14.",  
          "rejected_action": { "type": "CAST_SPELL", "seq_num": 14 } }  
  
-- Server re-issues the current PRIORITY_GRANT so P1 can try again --  
S->P1   { "type": "PRIORITY_GRANT", "seq_num": 16,  
          "player_id": "player_1", "time_limit_ms": 60000 }
```

## 6. Game Lifecycle

### 6.1. Overview

From the server's perspective, a game progresses through five top-level states:

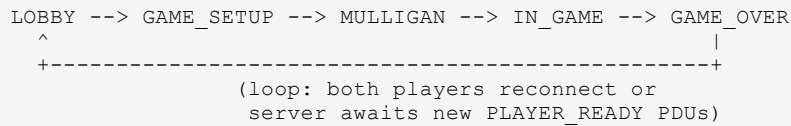


Figure 3: Game Lifecycle State Machine

The server **MUST** process these states in the order shown. After broadcasting `GAME_OVER`, the server **MUST** return to the `LOBBY` state and await new `PLAYER_READY` PDUs on the same TCP connections, allowing the same two players to start a new game without reconnecting. Any state **MUST** transition immediately to `GAME_OVER` if a player disconnects and fails to reconnect within the implementation-defined timeout.

### 6.2. LOBBY State

The server enters the `LOBBY` state upon startup and also re-enters it after every `GAME_OVER`. In `LOBBY`, the server awaits a fresh `PLAYER_READY` PDU from each connected player. TCP connections established at startup are reused for subsequent games; the server **MUST NOT** close connections at `GAME_OVER` unless a TCP-level error or heartbeat timeout has occurred. The server responds to each `PLAYER_READY` with a `GAME_STATE_UPDATE` reflecting the current lobby status.

The `player_id` field in `PLAYER_READY` is client-chosen and **MUST** be a non-empty string. The server **MUST** reject a `PLAYER_READY` whose `player_id` is already claimed by the other connected player, responding with `ERROR` code `DUPLICATE_ID`. Player IDs are reset at the start of each `LOBBY` state, so the same ID **MAY** be reused across games.

`PLAYER_READY` is exempt from the priority-echo `seq_num` rule. Its `seq_num` **MUST** be a client-maintained counter starting at 1 and incrementing with each `PLAYER_READY` sent. The server does not validate the `PLAYER_READY` `seq_num` against any priority token; it **MAY** use it solely for duplicate-detection or logging. A player **MAY** send a subsequent `PLAYER_READY` in the `LOBBY` state before both players are ready; the server **MUST** replace the earlier submission with the new deck list and respond with an updated `GAME_STATE_UPDATE`.

```

-- Player 1 connects and declares their deck (8 cards shown, up to 50 allowed) --

C->S    { "type": "PLAYER_READY", "seq_num": 1, "player_id": "player_1",
          "deck_list": [
            "lightning_bolt_001", "lightning_bolt_002", "lightning_bolt_003",
            "shock_001", "shock_002", "goblin_guide_001",
            "mountain_001", "mountain_002"
          ]
        }

```

```
        // ... up to 50 cards total
    ] }

S->P1    { "type": "GAME_STATE_UPDATE", "seq_num": 2, "state": {
            "phase": "LOBBY", "players_ready": 1, "waiting_for": ["player_2"]
          } }

-- Invalid deck: too many cards --
C->S      { "type": "PLAYER_READY", "seq_num": 1, "player_id": "player_1",
            "deck_list": [ /* 51 cards */ ] }
S->P1     { "type": "ERROR", "seq_num": 1, "code": "ILLEGAL_DECK",
            "message": "Deck contains 51 cards; maximum is 50." }
```

Transition: When both players have sent a valid `PLAYER_READY` PDU, the server transitions to `GAME_SETUP`.

### 6.3. `GAME_SETUP` State

The server performs the following operations automatically, without requiring player input:

1. Validate both deck lists (1 to 50 cards; legal cards from the fixed set only). The server **MUST** reject a `PLAYER_READY` PDU whose `deck_list` is empty or contains more than 50 entries, responding with `ERROR` code `ILLEGAL_DECK`.
2. Initialize each player's life total to 20.
3. Shuffle each player's deck using a server-side random number generator.
4. Draw seven cards for each player.
5. Determine which player goes first via a random coin flip.
6. Broadcast a personalized `GAME_STATE_UPDATE` to each player containing the initial life totals, hand, and library count.

```
-- Server broadcasts initial state after setup (shown for Player 1) --

S->P1    { "type": "GAME_STATE_UPDATE", "seq_num": 3, "state": {
            "turn": 0, "phase": "MULLIGAN", "active_player": "player_1",
            "life_totals": { "player_1": 20, "player_2": 20 },
            "hand":         [ "lightning_bolt_001", "shock_001", "mountain_001",
                              "mountain_002", "goblin_guide_001",
                              "lightning_bolt_002", "mountain_003" ],
            "hand_counts":  { "player_2": 7 },
            "library_counts": { "player_1": 43, "player_2": 43 },
            "battlefield":   { "player_1": [], "player_2": [] },
            "graveyard":     { "player_1": [], "player_2": [] },
            "stack":         []
          } }
```

**NOTE:** Life totals **MUST** be set to 20 before the initial `GAME_STATE_UPDATE` is broadcast. The server **MUST NOT** begin the first turn until both players have completed the mulligan phase.

Transition: Immediately after setup completes, the server transitions to `MULLIGAN`.



## 6.4. MULLIGAN State

Each player independently decides whether to keep their opening hand or take a mulligan. MTGNP uses the London Mulligan rule: a player who mulligans draws a new hand of seven cards, then puts a number of cards on the bottom of their library equal to the number of times they have mulliganed.

Each player **MUST** send a MULLIGAN\_CHOICE PDU. If keep is false, the server redraws and sends a new GAME\_STATE\_UPDATE to that player. A player **MAY** mulligan multiple times with no protocol-imposed minimum hand size. When keep is true and the player has mulliganed N times, the cards\_to\_bottom array **MUST** contain exactly N card IDs from the player's current hand. The server **MUST** validate this count and **MUST** respond with ERROR code ILLEGAL\_ACTION if the array length does not match the mulligan count or contains cards not in the player's hand.

```
-- Player keeps their opening hand (seq_num 3 echoes the setup GAME_STATE_UPDATE) --
C->S    { "type": "MULLIGAN_CHOICE", "seq_num": 3, "keep": true, "cards_to_bottom": [] }

-- Player mulligans instead (seq_num 3 echoes the same setup GAME_STATE_UPDATE) --
C->S    { "type": "MULLIGAN_CHOICE", "seq_num": 3, "keep": false, "cards_to_bottom": [] }
S->C    { "type": "GAME_STATE_UPDATE", "seq_num": 4, ... } // new 7-card hand sent

-- Player keeps after mulligan (echoes redraw GAME_STATE_UPDATE seq_num; must bottom 1
card) --
C->S    { "type": "MULLIGAN_CHOICE", "seq_num": 4, "keep": true,
        "cards_to_bottom": ["lightning_bolt_002"] }
```

Transition: When both players have sent MULLIGAN\_CHOICE with keep: true, the server transitions to IN\_GAME and begins the first player's turn.

## 6.5. IN\_GAME State

The IN\_GAME state encompasses the full turn loop described in Sections 7 through 9. The server cycles through turns, alternating the Active Player, until a win/loss condition is detected. The turn counter **MUST** be set to 1 when IN\_GAME begins (the first player's first turn is turn 1). The server increments the turn counter at the end of each Cleanup Step before beginning the next player's Untap Step. Win conditions that **MUST** trigger an immediate transition to GAME\_OVER are:

- A player's life total reaches zero or below.
- A player is required to draw a card from an empty library.
- A player sends a CONCEDE PDU.
- A player's connection is lost and the reconnect timer expires.

## 6.6. GAME\_OVER State

The server broadcasts a GAME\_OVER PDU to all connected players, then immediately transitions back to LOBBY state. The existing TCP connections are retained. Both players **MUST** send a fresh PLAYER\_READY PDU to begin a new game. The reason field in GAME\_OVER **MUST** be one of: LIFE\_ZERO, DECK\_EMPTY, CONCEDE, or DISCONNECT. In all cases, winner\_id **MUST** be set to the surviving or non-offending player: for LIFE\_ZERO and DECK\_EMPTY the winner is the player who did

not trigger the condition; for CONCEDE the winner is the player who did not concede; for DISCONNECT the winner is the player who remained connected. If a player disconnects at this point and fails to reconnect within the implementation-defined timeout, the server MAY close that connection and re-enter a waiting state.

## 7. Turn Structure

### 7.1. Overview

Each turn consists of a fixed sequence of phases and steps. Steps that open a priority window are marked below. Unmarked steps transition automatically without player input.

```
UNTAP STEP
|
UPKEEP STEP      <-- priority window
|
DRAW STEP        <-- priority window
|
PRECOMBAT MAIN PHASE <-- priority window (sorcery speed for AP)
|
COMBAT PHASE      <-- see Section 9 for sub-steps
|
POSTCOMBAT MAIN PHASE <-- priority window (sorcery speed for AP)
|
END STEP          <-- priority window
|
CLEANUP STEP
```

Figure 4: Turn Phase Sequence

### 7.2. Untap Step

At the start of each turn, the server broadcasts PHASE\_TRANSITION with to\_phase: "UNTAP". The server then untaps all permanents controlled by the Active Player and resets land\_played\_this\_turn to false for the new Active Player. The server broadcasts a GAME\_STATE\_UPDATE to both players reflecting the updated tapped state. No priority is given to either player during this step. The server MUST then broadcast PHASE\_TRANSITION with to\_phase: "UPKEEP" and transition immediately.

### 7.3. Upkeep Step

The server broadcasts PHASE\_TRANSITION with to\_phase: "UPKEEP" and opens a priority window with the Active Player receiving priority first. Both players may cast instants and activate abilities. The step ends when both players consecutively pass priority with an empty stack (see Section 8).

### 7.4. Draw Step

The server broadcasts PHASE\_TRANSITION with to\_phase: "DRAW" at the start of this step. The server then draws one card for the Active Player and sends a personalized GAME\_STATE\_UPDATE, followed

by a priority window. Note: on the very first turn of the game, the first player does NOT draw a card; the server still broadcasts PHASE\_TRANSITION to DRAW and still opens a priority window, but no card is added to the hand.

## 7.5. Main Phases

There are two Main Phases: Precombat (before combat) and Postcombat (after combat). The server broadcasts PHASE\_TRANSITION (to\_phase: "PRECOMBAT\_MAIN" or "POSTCOMBAT\_MAIN" as appropriate) at the start of each. During Main Phases, the Active Player MAY cast sorceries, creatures, enchantments, and artifacts, and play one land per turn (playing a land does not use the stack and does not require priority). After a land is played, the Active Player retains priority; the server broadcasts an updated GAME\_STATE\_UPDATE and then re-issues PRIORITY\_GRANT to the Active Player. Both players MAY cast instants at any time they hold priority.

Mana Abilities: Activating a mana ability (e.g., tapping a land or creature for mana) does not use the stack and does not require priority. In MTGNP 1.0, mana production is handled implicitly: the client declares the full mana\_payment in the CAST\_SPELL or ACTIVATE\_ABILITY PDU, and the server deducts the corresponding mana sources from the game state in a single atomic step. No separate PDU is defined for mana ability activation. The server MUST respond with ERROR code INSUFFICIENT\_MANA if the declared payment cannot be satisfied by the player's available mana sources.

## 7.6. Combat Phase

See Section 9 for the detailed Combat Phase sub-state machine.

## 7.7. End Step

The server broadcasts PHASE\_TRANSITION with to\_phase: "END\_STEP". A priority window then opens. Both players may cast instants and activate abilities. The step ends when both players consecutively pass priority with an empty stack.

## 7.8. Cleanup Step

The server broadcasts PHASE\_TRANSITION with to\_phase: "CLEANUP" at the start of this step. The server first checks whether the Active Player holds more than seven cards. If so, the server MUST send a GAME\_STATE\_UPDATE to the Active Player reflecting the current hand, then await a DISCARD PDU listing the cards to discard until hand size is seven or fewer. A DISCARD PDU whose card\_ids contains a card not in the Active Player's hand MUST be rejected with ERROR code ILLEGAL\_ACTION. After each valid DISCARD, the server broadcasts an updated GAME\_STATE\_UPDATE reflecting the reduced hand; if the hand still exceeds seven cards, the server awaits another DISCARD PDU (the client echoes the seq\_num of the most recently received GAME\_STATE\_UPDATE for each subsequent PDU). The server MUST NOT proceed until a valid DISCARD PDU brings the hand to seven or fewer cards. After discarding, the server removes all damage from creatures and clears any "until end of turn" effects, then broadcasts a GAME\_STATE\_UPDATE to both players reflecting the cleared state. No priority is given (in

MTGNP 1.0, no triggers fire at cleanup). The server increments the turn counter, switches the Active Player, and immediately begins the next turn's Untap Step.

## 8. Priority and the Stack

### 8.1. Priority Rules

The following rules govern priority in MTGNP:

1. At the start of each step that grants a priority window, the Active Player receives priority first.
2. A player who holds priority MAY cast a spell, activate a non-mana ability, or pass priority to the other player.
3. When a player casts a spell or activates an ability, the item is placed on the Stack and that player retains priority.
4. When a player passes priority, the opposing player receives priority.
5. When both players pass priority consecutively with a non-empty Stack, the server resolves the top Stack item (see Section 8.4). The Active Player then receives priority again.
6. When both players pass priority consecutively with an empty Stack, the current step ends and the server transitions to the next step.

### 8.2. Priority State Machine

The server maintains the following internal state machine for each priority window:

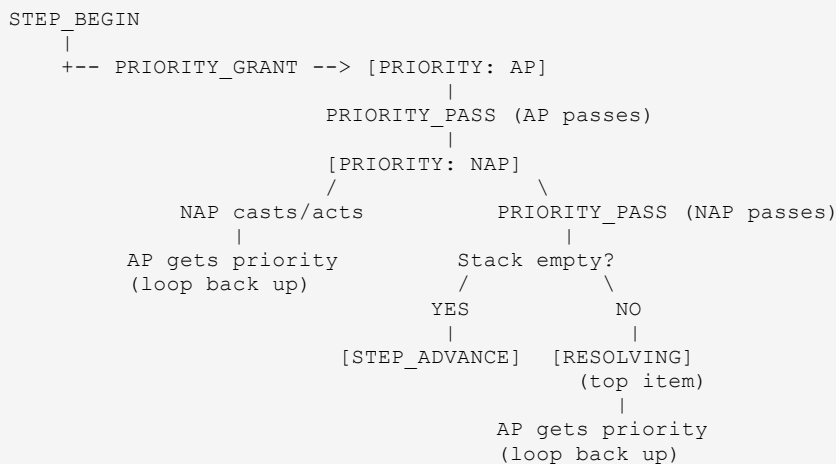


Figure 5: Priority State Machine

```

-- Full priority exchange: AP casts, NAP responds, both pass, spell resolves --

S->P1 { "type": "PRIORITY_GRANT", "player_id": "player_1", "seq_num": 5 }
C->S  { "type": "CAST_SPELL",      "seq_num": 5, "card_id": "shock_001",
        "targets": ["player_2"], "mana_payment": { "R": 1 } }
S->ALL { "type": "STACK_PUSH", "seq_num": 6, "stack_item_id": "stk_01", ... }

```

```
S->P1    { "type": "PRIORITY_GRANT", "player_id": "player_1", "seq_num": 6 }

C->S     { "type": "PRIORITY_PASS", "seq_num": 6 } // AP passes
S->P2    { "type": "PRIORITY_GRANT", "player_id": "player_2", "seq_num": 7 }

C->S     { "type": "PRIORITY_PASS", "seq_num": 7 } // NAP passes
// Both passed; stack non-empty -> RESOLVE stk_01
S->ALL   { "type": "STACK_RESOLVE", "seq_num": 8, "stack_item_id": "stk_01", "result":
"RESOLVED", ... }
S->P1    { "type": "PRIORITY_GRANT", "player_id": "player_1", "seq_num": 9 }

C->S     { "type": "PRIORITY_PASS", "seq_num": 9 } // AP passes (stack empty)
S->P2    { "type": "PRIORITY_GRANT", "player_id": "player_2", "seq_num": 10 }
C->S     { "type": "PRIORITY_PASS", "seq_num": 10 } // NAP passes (stack empty)
// Both passed with empty stack -> STEP ADVANCE
S->ALL   { "type": "PHASE_TRANSITION", "seq_num": 11, "from_phase": "UPKEEP", "to_phase":
"DRAW", ... }
```

### 8.3. The Stack

The Stack is a LIFO data structure maintained exclusively by the server. Each Stack item contains:

- `stack_item_id`: A server-assigned unique identifier.
- `item_type`: One of `SPELL`, `ABILITY`, or `TRIGGER_ABILITY`.
- `source_id`: The card or permanent that generated this item.
- `controller_id`: The player who cast or activated this item.
- `targets`: A list of target IDs (player IDs or permanent IDs).

The server broadcasts a `STACK_PUSH` PDU to both players whenever an item is added to the Stack, and a `STACK_RESOLVE` PDU whenever an item is resolved or fizzled. In the stack array of `GAME_STATE_UPDATE`, index 0 represents the bottom of the Stack (the oldest item, which resolves last); the final element represents the top of the Stack (the most recently added item, which resolves first).

```
-- Player 1 (AP) casts Lightning Bolt targeting Player 2 --

C->S     { "type": "CAST_SPELL", "seq_num": 7,
          "card_id": "lightning_bolt_001", "targets": ["player_2"],
          "mana_payment": { "R": 1 } }

S->ALL   { "type": "STACK_PUSH", "seq_num": 8, "stack_item_id": "stk_01",
          "item_type": "SPELL", "source": "lightning_bolt_001",
          "targets": ["player_2"], "controller": "player_1" }

// Stack now: [ stk_01 ] - AP retains priority
S->P1    { "type": "PRIORITY_GRANT", "player_id": "player_1",
          "seq_num": 8, "time_limit_ms": 60000 }
```

### 8.4. Stack Resolution

After every game event — including spell resolution, ability activation, land play, and phase transitions — the server **MUST** check for state-based actions (SBAs) before granting priority to any player. SBAs that **MUST** be checked include: a player whose life total is zero or less loses the game (`GAME_OVER` with reason `LIFE_ZERO`); a creature with toughness zero or less is moved to its owner's graveyard; a creature

with damage marked on it equal to or greater than its toughness is destroyed and moved to the graveyard. SBAs are applied repeatedly until none remain, then triggers from those events are placed on the Stack before priority is granted. If both players' life totals reach zero or less simultaneously (for example, from mutual combat damage), the Active Player loses and the Non-Active Player wins; the server broadcasts `GAME_OVER` with `winner_id` set to the Non-Active Player.

When both players consecutively pass priority with a non-empty Stack:

1. The server pops the top item from the Stack.
2. The server checks whether all targets are still legal. If all targets are illegal, the item fizzles with no effect; the server broadcasts `STACK_RESOLVE` with result: `FIZZLE`.
3. If targets are legal, the server applies the effect, broadcasts `STACK_RESOLVE` with result: `RESOLVED` and a `state_changes` array describing the effect, and then broadcasts `GAME_STATE_UPDATE` to both players reflecting the new game state.
4. The server grants priority to the Active Player. Steps 1-4 repeat until the Stack is empty.

```
-- Lightning Bolt resolves; server broadcasts result then re-grants priority --  
  
S->ALL { "type": "STACK_RESOLVE", "seq_num": 11, "stack_item_id": "stk_01",  
         "result": "RESOLVED",  
         "state_changes": [  
           { "type": "DAMAGE", "target": "player_2", "amount": 3 }  
         ] }  
  
S->ALL { "type": "GAME_STATE_UPDATE", "seq_num": 12, ... } // updated life totals  
S->P1  { "type": "PRIORITY_GRANT", "player_id": "player_1",  
         "seq_num": 12, "time_limit_ms": 60000 }  
  
-- If a target becomes illegal before resolution, the spell fizzles --  
S->ALL { "type": "STACK_RESOLVE", "seq_num": 13, "stack_item_id": "stk_03",  
         "result": "FIZZLE", "state_changes": [] }
```

## 8.5. Sequence Numbers

The `seq_num` field is defined in Section 5.4, which provides the normative description of its semantics for both client-to-server and server-to-client PDUs. In the context of priority, the server increments its `seq_num` counter with each PDU it sends. A client **MUST** echo this value in every action PDU submitted during that priority window. Actions carrying a mismatched `seq_num` are rejected with `ERROR` code `STALE_ACTION`.

## 8.6. Triggered Abilities

Triggered abilities are game actions that fire automatically in response to specific game events. They are identified by the keywords "When", "Whenever", or "At" at the start of their text. Once triggered, they are placed on the Stack and resolve like any other Stack item. Both players may respond to triggered abilities with instants and other abilities.

### 8.6.1. Trigger Detection

After every game event that may cause trigger conditions to be met, the server **MUST** check all triggered abilities on all permanents currently on the battlefield. Events that **MUST** trigger a check include, but are not limited to:

- A permanent enters the battlefield.
- A permanent leaves the battlefield (destroyed, exiled, bounced, or sacrificed).
- A creature dies (is moved to the graveyard from the battlefield).
- A spell or ability is cast.
- A player draws a card.
- A step or phase begins (e.g., "At the beginning of upkeep...").
- Combat damage is dealt.

If one or more triggered abilities fire as a result of a single event, the server **MUST** place all of them onto the Stack before granting priority to either player. Priority **MUST NOT** be granted until all pending trigger ordering decisions and optional trigger choices have been resolved (see Sections 8.6.2 and 8.6.3).

### 8.6.2. Trigger Ordering

When multiple triggered abilities fire simultaneously, the server places them on the Stack according to the following rules:

1. All triggers controlled by the Active Player are placed on the Stack first (and thus resolve last, being at the bottom).
2. All triggers controlled by the Non-Active Player are placed on top (and resolve first).
3. If a single player controls two or more triggers that fired simultaneously, the server **MUST** send a TRIGGER\_ORDER PDU to that player requesting the placement order. The player **MUST** respond with a TRIGGER\_ORDER\_RESPONSE PDU listing the trigger IDs in their preferred order before the Stack is updated.

```
-- Two triggers fire simultaneously for Player 1; server asks for order --  
  
S->P1 { "type": "TRIGGER_ORDER", "seq_num": 15, "player_id": "player_1",  
        "trigger_ids": ["trg_03", "trg_04"] }  
  
-- Player wants trg_04 on stack first (so trg_03 resolves first) --  
C->S { "type": "TRIGGER_ORDER_RESPONSE", "seq_num": 15,  
        "ordered_trigger_ids": ["trg_04", "trg_03"] }  
  
S->ALL { "type": "STACK_PUSH", "seq_num": 16, "stack_item_id": "stk_06", ... } // trg_04  
S->ALL { "type": "STACK_PUSH", "seq_num": 17, "stack_item_id": "stk_07", ... } // trg_03  
(on top)
```

**NOTE:** TRIGGER\_ORDER does not consume the player's priority — it is a mandatory ordering decision that is resolved before the Stack is updated and before any PRIORITY\_GRANT is issued.

### 8.6.3. Optional Triggers

Some triggered abilities use the phrasing "you may", giving the controlling player a choice of whether to put the ability on the Stack. When such a trigger fires, the server **MUST** send a TRIGGER\_CHOICE PDU

to the controlling player. The player responds with TRIGGER\_CHOICE\_RESPONSE containing an accept boolean. If accept is false, the trigger is discarded with no effect and no STACK\_PUSH is broadcast.

```
-- A 'you may' triggered ability fires; server asks the controller --

S->P1 { "type": "TRIGGER_CHOICE", "seq_num": 20,
        "trigger_id": "trg_02",
        "source_id": "gray_merchant_001",
        "effect_summary": "You may gain life equal to your devotion to black.",
        "targets": []
      }

-- Player accepts --
C->S { "type": "TRIGGER_CHOICE_RESPONSE", "seq_num": 20, "trigger_id": "trg_02",
        "accept": true, "chosen_target": null }

S->ALL { "type": "STACK_PUSH", "seq_num": 21, "stack_item_id": "stk_05",
        "item_type": "TRIGGER_ABILITY", "source": "gray_merchant_001",
        "targets": [], "controller": "player_1" }

-- Player declines --
C->S { "type": "TRIGGER_CHOICE_RESPONSE", "seq_num": 20, "trigger_id": "trg_02",
        "accept": false }
// no STACK_PUSH broadcast; trigger is silently discarded
```

Like TRIGGER\_ORDER, TRIGGER\_CHOICE resolution is mandatory and MUST be completed before priority is granted after any game event.

#### 8.6.4. Triggered Abilities and the Stack

Once placed on the Stack, triggered abilities behave identically to spells: they may be responded to and they resolve from the top of the Stack downward. The server broadcasts a STACK\_PUSH PDU for each triggered ability placed on the Stack, with type: TRIGGER\_ABILITY, including the source permanent ID and any targets chosen at trigger resolution time.

When a triggered ability that requires a target fires, the server MUST send a TRIGGER\_CHOICE PDU to the controlling player asking them to choose a legal target before the STACK\_PUSH is broadcast. If no legal targets exist, the trigger is discarded immediately with no effect.

When a triggered ability resolves, the server applies its effect, broadcasts STACK\_RESOLVE, and grants priority to the Active Player, exactly as described in Section 8.4.



## 9. Combat Phase

### 9.1. Overview

The Combat Phase is a sub-state machine within IN\_GAME. It consists of up to six steps, each with its own priority window.

```
BEGIN_COMBAT
|
DECLARE_ATTACKERS    <-- AP declares; priority window follows
|
DECLARE_BLOCKERS     <-- NAP assigns blockers; priority window follows
|
ASSIGN_DAMAGE_ORDER  <-- AP orders multi-blockers; priority window
|
[FIRST_STRIKE_DAMAGE]<-- OPTIONAL: only if first/double strike present
|
COMBAT_DAMAGE        <-- server resolves damage; priority window
|
END_OF_COMBAT        <-- priority window; combat concludes
```

Figure 6: Combat Phase Sub-State Machine

### 9.2. Beginning of Combat Step

The server broadcasts PHASE\_TRANSITION with to\_phase: "BEGIN\_COMBAT" and opens a priority window. This is the last opportunity for either player to act before attackers are declared. Transition occurs after both players pass with an empty stack.

### 9.3. Declare Attackers Step

After the priority window in the Beginning of Combat Step closes, the server broadcasts a PHASE\_TRANSITION to DECLARE\_ATTACKERS. This transition implicitly signals the Active Player to send a DECLARE\_ATTACKERS PDU; no separate request PDU is defined. The Active Player lists all creatures they wish to attack with and their respective targets (the opposing player). An empty attackers array is legal and means no attack.

If no attackers are declared, the server MUST skip directly to the End of Combat Step. Tapped creatures and creatures with summoning sickness MUST NOT be declared as attackers; the server MUST validate and reject violations with ERROR code ILLEGAL\_ACTION. Declaring a creature as an attacker taps it immediately; the GAME\_STATE\_UPDATE broadcast after a valid DECLARE\_ATTACKERS PDU MUST reflect the updated tapped state of each declared attacker.

After a valid declaration, the server broadcasts GAME\_STATE\_UPDATE and opens a priority window.

### 9.4. Declare Blockers Step

After the priority window following attacker declaration closes, the server broadcasts a PHASE\_TRANSITION to DECLARE\_BLOCKERS. This transition implicitly signals the Non-Active Player to send a DECLARE\_BLOCKERS PDU; no separate request PDU is defined. The Non-Active Player lists which untapped creatures block which attacking creatures. A single creature may block only one attacker; multiple creatures may block the same attacker. Blocking does not cause blocking creatures to tap; their tapped state is unchanged by the act of blocking.

After a valid declaration, the server broadcasts GAME\_STATE\_UPDATE and opens a priority window.

```
-- AP declares two attackers (seq_num=22 from prior PRIORITY_GRANT) --

C->S    { "type": "DECLARE_ATTACKERS", "seq_num": 22,
          "attackers": [
            { "creature_id": "goblin_guide_001", "target": "player_2" },
            { "creature_id": "reckless_wurm_003", "target": "player_2" }
          ] }

S->ALL  { "type": "GAME_STATE_UPDATE", "seq_num": 23, ... } // updated battlefield state
S->P1   { "type": "PRIORITY_GRANT", "player_id": "player_1",
          "seq_num": 23, "time_limit_ms": 60000 }
```

## 9.5. Assign Damage Order Step

After the priority window following blocker declaration closes, the server broadcasts a PHASE\_TRANSITION to ASSIGN\_DAMAGE\_ORDER if and only if at least one attacker is blocked by two or more creatures. This transition implicitly signals the Active Player to send one ASSIGN\_DAMAGE\_ORDER PDU per multiply-blocked attacker. The Active Player specifies the order in which each such attacker assigns its combat damage among its blockers. After all orderings have been received, the server opens a final priority window before proceeding to the damage step. If no attacker is multiply-blocked, this step is skipped and the server advances directly to the First Strike Damage Step or Combat Damage Step.

## 9.6. First Strike Damage Step (Optional)

This step occurs only if at least one attacking or blocking creature has first strike or double strike. The server broadcasts PHASE\_TRANSITION with to\_phase: "FIRST\_STRIKE\_DAMAGE" and then resolves first-strike damage for those creatures only. The server then checks for state-based actions (creatures with lethal damage are moved to the graveyard), broadcasts an updated GAME\_STATE\_UPDATE, and opens a priority window before proceeding to the regular Combat Damage Step.

## 9.7. Combat Damage Step

MTGNP 1.0 does not implement trample. A blocked attacker deals its full combat damage to its blocker(s) only, never to the defending player. An unblocked attacker deals damage equal to its power directly to the defending player. The server broadcasts PHASE\_TRANSITION with to\_phase: "COMBAT\_DAMAGE" and then simultaneously assigns combat damage from all attacking and blocking creatures, excluding creatures with first strike (but NOT double strike) that already dealt damage in the First Strike Damage

Step. Double-strike creatures deal damage in both steps. The server applies all damage, updates life totals, moves creatures with lethal damage to the graveyard, and checks win conditions. It then broadcasts COMBAT\_DAMAGE\_RESULT, sends a personalized GAME\_STATE\_UPDATE to each player, and broadcasts PHASE\_TRANSITION to END\_OF\_COMBAT; the priority window for this step is opened as described in Section 9.8.

```
-- Server resolves combat damage and broadcasts result --

S->ALL {
  "type": "COMBAT_DAMAGE_RESULT",
  "seq_num": 27,
  "damage_events": [
    { "source": "grizzly_bears_001", "target": "player_2", "amount": 2 },
    { "source": "wall_of_stone_004", "target": "grizzly_bears_001", "amount": 3 }
  ],
  "life_totals": { "player_1": 20, "player_2": 18 },
  "creatures_died": ["grizzly_bears_001"]
}

S->P1 { "type": "GAME_STATE_UPDATE", "seq_num": 28, ... } // updated state
(personalized for P1)
S->P2 { "type": "GAME_STATE_UPDATE", "seq_num": 29, ... } // updated state
(personalized for P2)

S->ALL { "type": "PHASE_TRANSITION", "seq_num": 30, "from_phase": "COMBAT_DAMAGE",
  "to_phase": "END_OF_COMBAT", "active_player": "player_1" }

S->P1 { "type": "PRIORITY_GRANT", "player_id": "player_1",
  "seq_num": 31, "time_limit_ms": 60000 }
```

## 9.8. End of Combat Step

The server broadcasts PHASE\_TRANSITION with to\_phase: "END\_OF\_COMBAT" and opens a priority window. After both players pass priority with an empty stack, the server clears all combat-related state (attacker/blocker assignments, combat damage marked on permanents) and broadcasts PHASE\_TRANSITION with to\_phase: "POSTCOMBAT\_MAIN" to begin the Postcombat Main Phase.

## 10. PDU Reference

### 10.1. PDU Summary

The following table lists all PDUs defined in this document. Direction abbreviations: C->S = Client to Server; S->C = Server to one Client; S->ALL = Server broadcast to both Clients.

| Message Type      | Dir  | Phase | Key Fields             | Notes                                                      |
|-------------------|------|-------|------------------------|------------------------------------------------------------|
| PLAYER_READY      | C->S | Lobby | player_id, deck_list[] | 1-50 cards; server rejects invalid decks with ILLEGAL_DECK |
| GAME_STATE_UPDATE | S->C | All   | visible_state, seq_num | Personalized per player; hidden info filtered out          |

| Message Type            | Dir    | Phase    | Key Fields                                                              | Notes                                                     |
|-------------------------|--------|----------|-------------------------------------------------------------------------|-----------------------------------------------------------|
| MULLIGAN_CHOICE         | C->S   | Setup    | keep: bool, cards_to_bottom[], seq_num                                  | Server redraws if keep is false (London Mulligan)         |
| PHASE_TRANSITION        | S->ALL | All      | from_phase, to_phase, active_player                                     | Broadcast when server advances a step or phase            |
| PRIORITY_GRANT          | S->C   | Priority | player_id, seq_num, time_limit_ms                                       | Sent only to the player who now holds priority            |
| PRIORITY_PASS           | C->S   | Priority | seq_num                                                                 | seq_num must match current priority token                 |
| CAST_SPELL              | C->S   | Priority | card_id, targets[], mana_payment, seq_num                               | Server validates; pushes to stack on success              |
| ACTIVATE_ABILITY        | C->S   | Priority | source_id, ability_index, targets[], seq_num                            | Mana abilities bypass the stack entirely                  |
| STACK_PUSH              | S->ALL | Stack    | stack_item_id, item_type, source, targets[]                             | item_type: SPELL   ABILITY   TRIGGER_ABILITY              |
| TRIGGER_ORDER           | S->C   | Stack    | player_id, trigger_ids[]                                                | Player must specify order for their simultaneous triggers |
| TRIGGER_ORDER_RESPONSE  | C->S   | Stack    | ordered_trigger_ids[]                                                   | Triggers listed in desired stack placement order          |
| TRIGGER_CHOICE          | S->C   | Stack    | trigger_id, source_id, effect_summary, legal_targets[], requires_target | Ask player to accept optional trigger or choose a target  |
| TRIGGER_CHOICE_RESPONSE | C->S   | Stack    | trigger_id, accept: bool, chosen_target?                                | accept=false discards the trigger with no effect          |
| STACK_RESOLVE           | S->ALL | Stack    | stack_item_id, result, state_changes[]                                  | result: RESOLVED or FIZZLE                                |
| DECLARE_ATTACKERS       | C->S   | Combat   | attackers[]: {creature_id, target}                                      | Empty array = no attack (still required)                  |
| DECLARE_BLOCKERS        | C->S   | Combat   | blockers[]: {creature_id, blocking_id}                                  | Server validates legality of each block                   |
| ASSIGN_DAMAGE_ORDER     | C->S   | Combat   | attacker_id, blocker_order[]                                            | Required when multiple blockers on one attacker           |
| COMBAT_DAMAGE_RESULT    | S->ALL | Combat   | damage_events[], life_totals, creatures_died[]                          | Server computes all damage simultaneously                 |
| PLAY_LAND               | C->S   | Main     | card_id, seq_num                                                        | Does not use the stack; one per turn limit                |
| DISCARD                 | C->S   | Cleanup  | card_ids[], seq_num                                                     | Required when hand size > 7 at cleanup                    |
| CONCEDE                 | C->S   | Any      | player_id, seq_num                                                      | Triggers immediate GAME_OVER                              |
| GAME_OVER               | S->ALL | End      | winner_id, loser_id, reason                                             | reason: LIFE_ZERO   DECK_EMPTY   CONCEDE   DISCONNECT     |
| ERROR                   | S->C   | Any      | code, message, rejected_action                                          | Game continues; rejected action is discarded              |

| Message Type | Dir  | Phase | Key Fields         | Notes                                 |
|--------------|------|-------|--------------------|---------------------------------------|
| PING         | C->S | Any   | timestamp, seq_num | Heartbeat — server responds with PONG |
| PONG         | S->C | Any   | timestamp          | Echo of the client's PING timestamp   |

## 10.2. PDU Schemas

The following subsections provide a complete JSON schema for every PDU defined in this document. Comments (`// ...`) are annotations only and are not part of the JSON encoding.

### 10.2.1. PLAYER\_READY (C->S)

```
{
  "type":      "PLAYER_READY",
  "seq_num":   1,           // monotonically increasing message counter
  "player_id": "player_1",  // client-chosen non-empty string; must be unique in
  // this lobby
  "deck_list": [           // 1 to 50 card IDs
    "lightning_bolt_001",
    "mountain_001",
    "goblin_guide_001"
    // ...
  ]
}
```

### 10.2.2. GAME\_STATE\_UPDATE (S->C)

GAME\_STATE\_UPDATE is used in two distinct contexts with different state object structures. During LOBBY, the state object contains lobby metadata. During all other phases (MULLIGAN, IN\_GAME), it contains the full game state as shown below.

```
// Lobby-phase variant (sent after each PLAYER_READY):
{
  "type":      "GAME_STATE_UPDATE",
  "seq_num":   2,
  "state": {
    "phase":      "LOBBY",
    "players_ready": 1, // how many players have sent PLAYER_READY
    "waiting_for": ["player_2"] // player_ids not yet ready
  }
}
```

```
// In-game variant (MULLIGAN and IN_GAME phases):
{
  "type":      "GAME_STATE_UPDATE",
  "seq_num":   44,
  "state": {
    "turn":      5,
    "active_player": "player_1",
    "phase":      "PRECOMBAT_MAIN",
    "priority_holder": "player_1", // null during UNTAP and CLEANUP steps
    "life_totals": { "player_1": 17, "player_2": 12 },
    "stack": [

```

```

    { "stack_item_id": "stk_01", "item_type": "SPELL",
      "source": "lightning_bolt_001", "targets": ["player_2"],
      "controller": "player_1" }
  ],
  "battlefield": {
    // Each permanent id matches its card instance id from the original deck_list
    "player_1": [ { "id": "mountain_001", "tapped": true } ], // Non-creatures: id and
    tapped only
    "player_2": [ { "id": "wall_of_stone_004", "tapped": false,
                    "damage": 0, "power": 0, "toughness": 8,
                    "summoning_sick": false } ] // Creatures add: damage, power,
    toughness, summoning_sick
  },
  "graveyard": { "player_1": [], "player_2": [] }, // ordered by insertion: index
0 = first card placed, last = most recently added
  "hand": { "player_1": ["shock_002", "forest_003"] },
  "hand_counts": { "player_2": 4 },
  "library_counts": { "player_1": 13, "player_2": 11 },
  "land_played_this_turn": true // true if AP has already played a land this turn
}
}

```

### 10.2.3. MULLIGAN\_CHOICE (C->S)

```

{
  "type": "MULLIGAN_CHOICE",
  "seq_num": 3, // monotonically increasing message counter
  "keep": true, // false = take a mulligan
  "cards_to_bottom": ["shock_001"] // must equal mulligan count when keep=true
}

```

### 10.2.4. PHASE\_TRANSITION (S->ALL)

```

{
  "type": "PHASE_TRANSITION",
  "seq_num": 10, // server-issued sequence number
  "from_phase": "UPKEEP",
  "to_phase": "DRAW",
  "active_player": "player_1",
  "turn": 3
}

```

The complete set of valid string values for from\_phase and to\_phase, in turn order, is:

|                     |                                                            |
|---------------------|------------------------------------------------------------|
| UNTAP               | - Untap Step (no priority; server transitions immediately) |
| UPKEEP              | - Upkeep Step                                              |
| DRAW                | - Draw Step                                                |
| PRECOMBAT_MAIN      | - Precombat Main Phase                                     |
| BEGIN_COMBAT        | - Beginning of Combat Step                                 |
| DECLARE_ATTACKERS   | - Declare Attackers Step                                   |
| DECLARE_BLOCKERS    | - Declare Blockers Step                                    |
| ASSIGN_DAMAGE_ORDER | - Assign Damage Order Step                                 |
| FIRST_STRIKE_DAMAGE | - First Strike Damage Step (optional)                      |
| COMBAT_DAMAGE       | - Combat Damage Step                                       |
| END_OF_COMBAT       | - End of Combat Step                                       |
| POSTCOMBAT_MAIN     | - Postcombat Main Phase                                    |
| END_STEP            | - End Step                                                 |
| CLEANUP             | - Cleanup Step                                             |

### 10.2.5. PRIORITY\_GRANT (S->C)

```
{
  "type":      "PRIORITY_GRANT",
  "player_id": "player_1",
  "seq_num":   43,
  "time_limit_ms": 60000           // server-enforced response deadline
}
```

### 10.2.6. PRIORITY\_PASS (C->S)

```
{
  "type":      "PRIORITY_PASS",
  "seq_num": 43           // must match current PRIORITY_GRANT seq_num
}
```

### 10.2.7. CAST\_SPELL (C->S)

```
{
  "type":      "CAST_SPELL",
  "seq_num":   7,
  "card_id":   "lightning_bolt_001",
  "targets":   ["player_2"], // empty array if spell has no targets
  "mana_payment": { "R": 1 } // color keys: W U B R G, generic key: "X"
}
```

### 10.2.8. ACTIVATE\_ABILITY (C->S)

```
{
  "type":      "ACTIVATE_ABILITY",
  "seq_num":   9,
  "source_id": "llanowar_elves_002",
  "ability_index": 0,           // 0-based index into permanent's ability list
  "targets":   [],
  "cost_payment": { "tap": true, "mana": {} } // tap: true only if ability requires tapping
  // Server rejects with ILLEGAL_ACTION if permanent is already tapped
}
```

### 10.2.9. STACK\_PUSH (S->ALL)

```
{
  "type":      "STACK_PUSH",
  "seq_num":   8,           // server-issued sequence number
  "stack_item_id": "stk_01",
  "item_type":  "SPELL",    // SPELL | ABILITY | TRIGGER_ABILITY
  "source":    "lightning_bolt_001",
  "targets":   ["player_2"],
  "controller": "player_1"
}
```

### 10.2.10. TRIGGER\_ORDER (S->C)

```
{
  "type":      "TRIGGER_ORDER",
  "seq_num":   15,         // server-issued sequence number
}
```

```
"player_id":    "player_1",
"trigger_ids":  ["trg_03", "trg_04"]  // player must order these
}
```

### 10.2.11. TRIGGER\_ORDER\_RESPONSE (C->S)

```
{
  "type":          "TRIGGER_ORDER_RESPONSE",
  "seq_num":       15,  // must match the corresponding TRIGGER_ORDER seq_num
  "ordered_trigger_ids": ["trg_04", "trg_03"]
  // trg_04 placed first (resolves last); trg_03 on top (resolves first)
}
```

### 10.2.12. TRIGGER\_CHOICE (S->C)

```
{
  "type":          "TRIGGER_CHOICE",
  "seq_num":       20,  // server-issued sequence number
  "trigger_id":     "trg_02",
  "source_id":      "gray_merchant_001",
  "effect_summary": "You may gain life equal to your devotion to black.",
  "requires_target": false,  // true if player must also pick a target
  "legal_targets":  []       // populated when requires_target is true;
                             // elements are player_id strings or permanent id
  strings
}
```

### 10.2.13. TRIGGER\_CHOICE\_RESPONSE (C->S)

```
{
  "type":          "TRIGGER_CHOICE_RESPONSE",
  "seq_num":       20,  // must match the corresponding TRIGGER_CHOICE seq_num
  "trigger_id":     "trg_02",
  "accept":         true,
  "chosen_target":  null  // non-null only when accept=true AND
requires_target=true;    // absent or null when accept=false or
                           requires_target=false
}
```

### 10.2.14. STACK\_RESOLVE (S->ALL)

```
{
  "type":          "STACK_RESOLVE",
  "seq_num":       31,  // server-issued sequence number
  "stack_item_id": "stk_01",
  "result":        "RESOLVED",  // RESOLVED | FIZZLE
  "state_changes": [
    { "change_type": "DAMAGE", "target": "player_2", "amount": 3 },
    { "change_type": "LIFE_GAIN", "target": "player_1", "amount": 2 },
    { "change_type": "DESTROY", "target": "wall_of_stone_004" }
  ]
}
```

### 10.2.15. DECLARE\_ATTACKERS (C->S)



```
{
  "type":      "DECLARE_ATTACKERS",
  "seq_num":   22,
  "attackers": [
    { "creature_id": "goblin_guide_001", "target": "player_2" },
    { "creature_id": "reckless_wurm_003", "target": "player_2" }
  ]
  // send empty attackers array to declare no attack
}
```

#### 10.2.16. DECLARE\_BLOCKERS (C->S)

```
{
  "type":      "DECLARE_BLOCKERS",
  "seq_num":   24,
  "blockers": [
    { "creature_id": "wall_of_stone_004", "blocking_id": "goblin_guide_001" }
  ]
  // send empty blockers array to not block
}
```

#### 10.2.17. ASSIGN\_DAMAGE\_ORDER (C->S)

```
{
  "type":      "ASSIGN_DAMAGE_ORDER",
  "seq_num":   26,
  "attacker_id": "reckless_wurm_003",
  "blocker_order": ["wall_of_stone_004", "grizzly_bears_002"]
  // damage assigned to wall first, overflow goes to bears
}
```

#### 10.2.18. COMBAT\_DAMAGE\_RESULT (S->ALL)

```
{
  "type": "COMBAT_DAMAGE_RESULT",
  "seq_num": 27,           // server-issued sequence number
  "damage_events": [
    { "source": "goblin_guide_001", "target": "player_2", "amount": 2 },
    { "source": "wall_of_stone_004", "target": "goblin_guide_001", "amount": 3 }
  ],
  "life_totals": { "player_1": 20, "player_2": 18 },
  "creatures_died": ["goblin_guide_001"]
}
```

#### 10.2.19. PLAY\_LAND (C->S)

```
{
  "type":      "PLAY_LAND",
  "seq_num":   5,
  "card_id": "mountain_003"
  // does not use the stack; one land play permitted per turn
}
```

#### 10.2.20. DISCARD (C->S)

```
{
```

```
{
  "type":      "DISCARD",
  "seq_num":   50,
  "card_ids": ["lightning_bolt_004", "shock_003"]
  // sent at cleanup when hand size exceeds 7
}
```

### 10.2.21. CONCEDE (C->S)

```
{
  "type":      "CONCEDE",
  "seq_num":   99,
  "player_id": "player_2"
}
```

### 10.2.22. GAME\_OVER (S->ALL)

```
{
  "type":      "GAME_OVER",
  "seq_num":   100,           // server-issued sequence number
  "winner_id": "player_1",
  "loser_id":  "player_2",
  "reason":    "LIFE_ZERO"
  // reason: LIFE_ZERO | DECK_EMPTY | CONCEDE | DISCONNECT
}
```

### 10.2.23. ERROR (S->C)

```
{
  "type":      "ERROR",
  "seq_num":   14,           // echoes the seq_num of the rejected action when
available
  "code":      "STALE_ACTION",
  "message":   "Priority token mismatch. Expected seq_num 16, got 14.",
  "rejected_action": { "type": "CAST_SPELL", "seq_num": 14, "card_id": "..."}
}
```

### 10.2.24. PING (C->S)

```
{
  "type":      "PING",
  "seq_num":   1,           // used to correlate with PONG response
  "timestamp": 1745000000000 // Unix epoch milliseconds
}
```

### 10.2.25. PONG (S->C)

```
{
  "type":      "PONG",
  "seq_num":   1,           // echoes the PING seq_num
  "timestamp": 1745000000000 // echoes the PING timestamp
}
```

## 11. Error Handling

When the server receives an invalid or illegal PDU from a client, it **MUST**:

1. Send an ERROR PDU to the originating client containing: an error code (see below), a human-readable message string, and a copy of the rejected action PDU.
2. Discard the illegal action and leave the game state unchanged.
3. If the player still holds priority, re-issue PRIORITY\_GRANT with the same seq\_num so the player may try again.

Defined error codes:

**INVALID\_JSON:** The received bytes could not be parsed as valid UTF-8 JSON.

**ILLEGAL\_DECK:** The submitted deck\_list is empty, contains more than 50 cards, or includes one or more cards not in the legal card set.

**UNKNOWN\_TYPE:** The type field does not match any known PDU type.

**STALE\_ACTION:** The seq\_num does not match the current priority token.

**NOT\_YOUR\_PRIORITY:** The client submitted an action PDU when it does not hold priority.

**ILLEGAL\_ACTION:** The action is syntactically valid but violates game rules (e.g., attacking with a tapped creature).

**ILLEGAL\_TARGET:** One or more targets in a CAST\_SPELL, ACTIVATE\_ABILITY, or TRIGGER\_CHOICE\_RESPONSE PDU are not legal targets.

**TRIGGER\_ORDER\_INVALID:** The TRIGGER\_ORDER\_RESPONSE does not contain exactly the trigger IDs that were sent in the corresponding TRIGGER\_ORDER PDU.

**TRIGGER\_CHOICE\_INVALID:** The TRIGGER\_CHOICE\_RESPONSE references an unknown trigger\_id, or chosen\_target is absent when a target is required.

**INSUFFICIENT\_MANA:** The mana\_payment provided does not satisfy the spell's mana cost.

**WRONG\_PHASE:** The action is not legal in the current phase (e.g., casting a sorcery outside a Main Phase).

**DUPLICATE\_ID:** The player\_id in a PLAYER\_READY PDU is already claimed by the other connected player in this lobby session.

**NOTE:** The server **MUST NOT** disconnect a client solely because it received an illegal action PDU. Disconnection **MUST** only occur on TCP-level errors or a heartbeat timeout.

## 12. Security Considerations

This document specifies a protocol intended for educational use in a controlled local network environment. The following security considerations are noted for completeness.

**Authentication:** MTGNP 1.0 does not define an authentication mechanism. Deployments where player identity matters SHOULD implement an application-layer authentication handshake before PLAYER\_READY is accepted.

**Confidentiality:** MTGNP 1.0 transmits all data, including player hand contents, as plaintext JSON over TCP. Deployments over untrusted networks SHOULD wrap TCP connections in TLS [RFC8446].

**Cheating Prevention:** Because all game logic resides on the server and every action is independently validated, a cheating client cannot force an illegal game state. The server MUST withhold hidden information (opponent hand contents) from GAME\_STATE\_UPDATE messages.

**Denial of Service:** A malicious client could stall the game by never sending required PDUs. Implementations SHOULD enforce the time\_limit\_ms field in PRIORITY\_GRANT. A player who does not respond within the time limit SHOULD be treated as disconnected.

## 13. References

### 13.1. Normative References

**[RFC2119]** Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", BCP 14, RFC 2119, March 1997.

**[RFC8259]** Bray, T., "The JavaScript Object Notation (JSON) Data Interchange Format", RFC 8259, December 2017.

**[RFC9293]** Eddy, W., "Transmission Control Protocol (TCP)", RFC 9293, August 2022.

### 13.2. Informative References

**[MTG-CR]** Wizards of the Coast, "Magic: The Gathering Comprehensive Rules", current edition.  
<https://magic.wizards.com/en/rules>

**[RFC8446]** Rescorla, E., "The Transport Layer Security (TLS) Protocol Version 1.3", RFC 8446, August 2018.

---

#### Author's Address

A. F. B. Laguna  
De La Salle University - Manila  
CSNETWK — Computer Networks  
Email: [ann.laguna@dlsu.edu.ph](mailto:ann.laguna@dlsu.edu.ph)

# Rubric

# Magic: The Gathering Multiplayer Network Protocol

## (MTGNP) — Machine Problem Rubric & Instructions

CSNETWK • T3 AY 2025–2026

---

### Objective

---

This machine problem requires you to implement a networked two-player card game system following the **Magic: The Gathering Multiplayer Network Protocol (MTGNP) v1.0**, as defined in RFC 0001 (CSNETWK). The primary learning goals are:

Magic: The Gathering is one of the most rules-dense games ever designed. This is intentional: the protocol reflects exactly the kind of layered, stateful, and specification-heavy system you will encounter when working with real-world network protocols. Successfully implementing MTGNP is not about memorising card game rules; it is about demonstrating that you can **read a formal specification and build something that faithfully follows it**. The same discipline applies to any protocol you implement in industry.

- Applying TCP socket programming to a real, non-trivial protocol specification.
- Reading and correctly implementing a formal RFC (including PDU structure, message framing, sequence number handling, and error codes)
- Building a client-server architecture where the server is the sole authoritative source of game state.
- Handling concurrent connections, heartbeat / keep-alive mechanics, and graceful disconnect/reconnect.

Your implementation will be evaluated both on protocol correctness and on your ability to explain the socket-level behaviour of your code during the demo.

### Deliverables

---

- Source code (server + client) in a single project repository or ZIP archive
  - I suggest using git for version control and keeping track of individual member's contributions but this is not required.
- README file in PDF containing:
  - Build and run instructions, including how to enable verbose mode
  - Work Distribution Matrix (see Groupings section)
  - AI Usage section — all tools used and how they were used
  - Any known limitations or deviations from the RFC

### Demo

---

**Deadline and Demo Date: 13th and 14th week** (exact date to be announced by your instructor). All group members must be present for the demo.

#### **Verbose Mode Prerequisite**

Your implementation must support a verbose mode that can be toggled on and off at runtime (e.g., via a command-line flag or startup argument). When enabled, verbose mode must print all PDUs sent and received, both client-side and server-side, to the console in a readable, clearly labelled format. **The MP will not be checked unless verbose mode is working and active during the demo. The MP will be an automatic zero.** Ensure both your client and server can be started in verbose mode before your checking slot.

### ⚠ Grading Deductions

- If any question relating to the submitted work, including implementation decisions, protocol behaviour, socket programming, or AI-generated content, is not answered properly during the demo, deductions in grades may be given for up to a grade of zero.
- The instructor may adjust individual grades based on group contribution.

## AI Usage Policy

Students **may** use AI tools (e.g., ChatGPT, GitHub Copilot, Claude) to assist with this machine problem. However, the following rules apply:

- You must document every AI tool used and describe how it was used in the README's AI Usage section.
- All AI-generated code must be reviewed, tested, and verified by the student before submission.
- The final submission must reflect your own understanding. Every member must be able to explain any part of the code — including AI-generated sections — during the demo.
- Blindly copying AI output without testing or comprehension is not allowed. Submissions that show a lack of understanding or appear to be largely untested AI output may receive reduced credit, up to a grade of zero.
- Reusing AI-generated content from another student's session or sharing AI outputs between groups is prohibited.

The goal of this policy is to let you use AI as a **learning aid**, not a shortcut. Understanding TCP sockets and protocol implementation is the core skill being assessed.

## Groupings

**Each group should have up to four members.** All members are expected to contribute meaningfully to the design, implementation, and testing of the project. Each participant must be capable of explaining all parts of the submission, even components they did not directly code.

A detailed report of tasks implemented by each member must be recorded in the README using the Work Distribution Matrix format below. Additional rows may be added as needed. **False reporting of work distribution is treated as academic misconduct.**

| Task / Feature                                      | Member 1 | Member 2 | Member 3 | Member 4 |
|-----------------------------------------------------|----------|----------|----------|----------|
| TCP Server: connection handling, framing, dispatch  |          |          |          |          |
| Game lifecycle: LOBBY, GAME_SETUP, MULLIGAN logic   |          |          |          |          |
| Turn & phase engine (all phases/steps, transitions) |          |          |          |          |
| Priority & Stack logic, spell/ability resolution    |          |          |          |          |
| Combat system (attackers, blockers, damage)         |          |          |          |          |
| Client implementation & state rendering             |          |          |          |          |

| Task / Feature                                            | Member 1 | Member 2 | Member 3 | Member 4 |
|-----------------------------------------------------------|----------|----------|----------|----------|
| PDU serialisation/deserialisation (all 25 PDU types)      |          |          |          |          |
| Error handling, PING/PONG heartbeat, disconnect logic     |          |          |          |          |
| Verbose mode (client + server PDU logging, toggle on/off) |          |          |          |          |
| Testing & interoperability                                |          |          |          |          |
| README / documentation / AI disclosure                    |          |          |          |          |

**Violations of academic integrity**, such as unauthorized sharing of code, plagiarism, or misrepresenting AI-generated work, may result in a grade of zero and may be referred to the appropriate academic disciplinary body. Code sharing between different groups is strictly prohibited. While discussion of ideas, clarification, and interoperability testing is encouraged, all source code must be the original work of the group submitting it. Any third-party libraries or tools must be cited appropriately.

#### Non-cooperating group members

If a member consistently fails to contribute

1. The group must first attempt to resolve the issue through clear, documented internal communication.
2. If unresolved, notify the course instructor with supporting evidence (e.g., chat logs, Git commits).
  - a. The group can decide to exclude non-contributing members.
  - b. Or the instructor may adjust individual grades.

## What Is and Is Not Allowed

The table below summarises academic integrity and tool-use policies for this machine problem.

| Activity                                               | ✓ Allowed                                                                    | ✗ Not Allowed                                                           |
|--------------------------------------------------------|------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| <b>Working with a group (2–4 people)</b>               | Yes – group work is supported                                                | Groups larger than 4                                                    |
| <b>Using AI tools (e.g., ChatGPT, Copilot, Claude)</b> | Yes – to understand TCP/socket concepts, generate boilerplate, or debug code | Submitting AI-generated code without review, testing, or acknowledgment |
| <b>Discussing protocol ideas with classmates</b>       | Yes – general discussion of MTGNP concepts is encouraged                     | Sharing source code or implementation files between groups              |
| <b>Testing with another group's client/server</b>      | Yes – for interoperability testing and debugging                             | Submitting shared or merged code from different groups                  |
| <b>Using open-source libraries/utilities</b>           | Yes – with proper citation in README                                         | Using uncredited third-party code                                       |
| <b>Submitting identical code as another group</b>      | No                                                                           | Plagiarism, even with minor changes                                     |



| Activity                                     | ✓ Allowed                                                                    | ✗ Not Allowed                                                    |
|----------------------------------------------|------------------------------------------------------------------------------|------------------------------------------------------------------|
| Individual understanding of group submission | Each member must be able to explain the entire solution                      | Relying solely on one member or AI without full team involvement |
| Acknowledging AI usage                       | Must include an AI Usage section in README describing all tools used and how | Omitting any mention of AI assistance                            |
| AI-generated code submitted as-is            | Only if reviewed, tested, and clearly acknowledged                           | Blindly copying AI output without comprehension or testing       |

## Grading Rubric (Total: 120/100 points)

Base points: 100; bonus: 20. Verbose mode must be working before any criterion is evaluated — see prerequisite row below. Even with a fully functioning code, the points are subject to deductions if the student cannot explain their work and answer questions.

| Category                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | Criterion                        | Poi<br>nts | Description                                                                                                                                                                                                                                                       |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>⚠ PREREQUISITE — Verbose Mode (checking will not proceed without this):</b> Both the client and server must support a verbose mode that can be toggled on and off (e.g., via a command-line flag or startup argument). When enabled, all PDUs sent and received — on both the client side and the server side — must be printed to the console in a readable, clearly labelled format. <b>The MP will not be checked unless verbose mode is working and active during the demo.</b> |                                  |            |                                                                                                                                                                                                                                                                   |
| TCP Sockets & Networking                                                                                                                                                                                                                                                                                                                                                                                                                                                               | TCP Server Setup & Client Accept | 10         | Correct TCP server socket creation on port 4444, binding, listening, and accepting exactly two client connections. Additional connections must be refused. Server must handle client disconnect/reconnect within the implementation-defined timeout.              |
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | Message Framing                  | 5          | All PDUs are correctly framed with a 4-byte big-endian length prefix followed by a valid UTF-8 JSON payload. Receiver reads exactly the indicated byte count before attempting to parse. PDUs must not exceed 65,535 bytes.                                       |
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | PDU Structure & seq_num          | 5          | Every PDU includes the required 'type' and 'seq_num' fields. Priority-bearing client PDUs echo the correct seq_num from the latest PRIORITY_GRANT or server request PDU. Server rejects stale seq_nums with ERROR code STALE_ACTION.                              |
| Game Lifecycle                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | LOBBY & PLAYER_READY Handling    | 10         | Server correctly enters LOBBY on startup and after each GAME_OVER. PLAYER_READY PDUs are validated: non-empty player_id, unique IDs, deck list of 1–50 cards from the fixed set. Server rejects duplicates with DUPLICATE_ID and invalid decks with ILLEGAL_DECK. |
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | GAME_SETUP & MULLIGAN            | 5          | Server initialises life totals at 20, shuffles decks, deals seven cards, and determines first player via random coin flip. London Mulligan rule: players who mulligan redraw a full hand and place N cards on the bottom when keeping after N mulligans.          |
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | IN_GAME Phase & Step Transitions | 10         | Server broadcasts PHASE_TRANSITION messages for all turn phases and steps (Untap, Upkeep, Draw, Main 1, Combat, Main                                                                                                                                              |

| Category                                 | Criterion                                      | Poi<br>nts                   | Description                                                                                                                                                                                                                                                                                                                                                                          |
|------------------------------------------|------------------------------------------------|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                          |                                                |                              | 2, End, Cleanup). Phase-specific rules are enforced — e.g., land drops only in Main Phase, no non-mana spells during Untap.                                                                                                                                                                                                                                                          |
|                                          | <b>GAME_OVER &amp; Session Restart</b>         | 5                            | Server correctly detects win/loss conditions (life total $\leq 0$ , empty library on draw, CONCEDE) and broadcasts GAME_OVER with the appropriate reason. Returns to LOBBY state and awaits fresh PLAYER_READY PDUs on the same TCP connections.                                                                                                                                     |
| <b>Server-Side Game Logic</b>            | <b>Game State Management &amp; Hidden Info</b> | 10                           | Server maintains the single authoritative Game State. GAME_STATE_UPDATE messages are personalised per player — each player's hand is hidden from the opponent. Library counts, battlefield, graveyard, and stack contents are correctly reflected.                                                                                                                                   |
|                                          | <b>Priority &amp; Stack Resolution</b>         | 10                           | Server correctly issues PRIORITY_GRANT to the appropriate player at each priority window. The stack (LIFO) is maintained correctly — spells and abilities push and resolve in order. Stack resolves when both players pass priority consecutively. STACK_PUSH and STACK_RESOLVE are broadcast. <b>At least 5 card effects (any) should be implemented to achieve the full point.</b> |
|                                          | <b>Combat System</b>                           | 10                           | Server correctly manages the full combat sequence: Declare Attackers, Declare Blockers, Assign Damage Order, optional First Strike Damage, and Combat Damage. Summoning sickness is enforced. COMBAT_DAMAGE_RESULT is broadcast with correct damage values.                                                                                                                          |
| <b>Client Implementation</b>             | <b>Client Sending &amp; State Rendering</b>    | 5                            | Client correctly sends all required PDU types (PLAYER_READY, MULLIGAN_CHOICE, CAST_SPELL, PLAY_LAND, PRIORITY_PASS, CONCEDE, etc.). Client accepts GAME_STATE_UPDATE as authoritative and renders visible state accurately, discarding any locally computed state.                                                                                                                   |
|                                          | <b>PING/PONG Heartbeat</b>                     | 5                            | Client sends PING PDUs at regular intervals (recommended: every 30 s) and disconnects if no PONG is received within the implementation-defined timeout (recommended: 10 s). Server responds to every PING with a matching PONG.                                                                                                                                                      |
| <b>Error Handling &amp; Code Quality</b> | <b>Error PDU Handling</b>                      | 5                            | Server sends ERROR PDUs with correct error codes (STALE_ACTION, ILLEGAL_ACTION, ILLEGAL_DECK, DUPLICATE_ID, etc.) in all cases specified by the RFC. Client handles ERROR PDUs gracefully without crashing.                                                                                                                                                                          |
|                                          | <b>Readability &amp; Comments</b>              | 5                            | Code is well-structured with meaningful variable and function names, and includes clear comments explaining non-obvious logic — especially socket setup, message framing, and protocol state transitions.                                                                                                                                                                            |
| <b>Bonus</b>                             | <b>Full Card Effects</b>                       | 10                           | Implementation of all Card Abilities and Effects.                                                                                                                                                                                                                                                                                                                                    |
|                                          | <b>Bonus Features</b>                          | 10                           | Additional features beyond the base specification: triggered-ability UI, spectator client, graphical interface, or other creative extensions. Must be demonstrated and explained during checking.                                                                                                                                                                                    |
| <b>TOTAL</b>                             |                                                | <b>100 points + 10 bonus</b> |                                                                                                                                                                                                                                                                                                                                                                                      |

## Instructor AI Disclaimer

---

The MTGNP RFC (RFC 0001, CSNETWK) is the primary work of the course instructor. AI tools — primarily Claude — was leveraged to assist with RFC formatting and JSON schema examples. All AI-generated content was thoroughly reviewed, validated, and adapted to meet the functional and educational goals of this course. The protocol design, game logic specification, and pedagogical structure reflect the instructor's original intent.

# Examples

# MTGNP RFC v3

Sample PDU Exchange — LOBBY + GAME\_SETUP + MULLIGAN + IN\_GAME  
+ GAME\_OVER

---

## 1. LOBBY State

---

Both players connect and declare their decks. The server waits until it has received a valid `PLAYER_READY` from each player before advancing.

### Step 1 - Player 1 sends `PLAYER_READY`

**C -> S**

```
{
  "type":      "PLAYER_READY",
  "seq_num":   1,
  "player_id": "player_1",
  "deck_list": [
    "lightning_bolt_001", "lightning_bolt_002", "lightning_bolt_003",
    "shock_001",           "shock_002",
    "goblin_guide_001",
    "mountain_001",       "mountain_002"
  ]
}
```

### Step 2 - Server acknowledges, waits for Player 2

**S -> P1**

```
{
  "type":      "GAME_STATE_UPDATE",
  "seq_num":   1,
  "state": {
    "phase":      "LOBBY",
    "players_ready": 1,
    "waiting_for": ["player_2"]
  }
}
```

### Step 3 - Player 2 sends `PLAYER_READY`

**C -> S**

```
{
  "type":      "PLAYER_READY",
  "seq_num":   1,
  "player_id": "player_2",
  "deck_list": [
    "counterspell_001", "counterspell_002",
  ]
}
```

```

    "gray_merchant_001", "gray_merchant_002",
    "island_001",        "island_002",
    "swamp_001",         "swamp_002"
  ]
}

```

## Step 4 - Server confirms both ready, transitions to GAME\_SETUP

**S -> ALL**

```

{
  "type":    "GAME_STATE_UPDATE",
  "seq_num": 2,
  "state": {
    "phase":    "GAME_SETUP",
    "players_ready": 2,
    "waiting_for": []
  }
}

```

## 2. GAME\_SETUP State

GAME\_SETUP is fully automatic — no client input is required. The server validates decks, sets life totals to 20, shuffles each deck, draws seven cards per player, and determines who goes first via coin flip. It then broadcasts a personalized GAME\_STATE\_UPDATE to each player before transitioning to MULLIGAN.

## Step 5 - Server sends personalized GAME\_STATE\_UPDATE to Player 1

Player 1's hand is visible to them; Player 2's hand is hidden (only the count is shown).

**S -> P1**

```

{
  "type":    "GAME_STATE_UPDATE",
  "seq_num": 3,
  "state": {
    "turn": 0, "phase": "MULLIGAN", "active_player": "player_1",
    "life_totals": { "player_1": 20, "player_2": 20 },
    "hand":
    ["lightning_bolt_001", "shock_001", "mountain_001", "mountain_002", "goblin_guide_001",
    "lightning_bolt_002", "mountain_003"],
    "hand_counts": { "player_2": 7 }, "library_counts": { "player_1": 1,
    "player_2": 1 },
    "battlefield": { "player_1": [], "player_2": [] },
    "graveyard":   { "player_1": [], "player_2": [] }, "stack": []
  }
}

```

## Step 6 - Server sends personalized GAME\_STATE\_UPDATE to Player 2

Player 2's hand is visible to them; Player 1's hand is hidden (only the count is shown).

## S -> P2

```
{
  "type": "GAME_STATE_UPDATE",
  "seq_num": 3,
  "state": {
    "turn": 0, "phase": "MULLIGAN", "active_player": "player_1",
    "life_totals": { "player_1": 20, "player_2": 20 },
    "hand":
["counterspell_001", "gray_merchant_001", "island_001", "swamp_001", "counterspell_002",
"gray_merchant_002", "swamp_002"],
    "hand_counts": { "player_1": 7 }, "library_counts": { "player_1": 1,
"player_2": 1 },
    "battlefield": { "player_1": [], "player_2": [] },
    "graveyard": { "player_1": [], "player_2": [] }, "stack": []
  }
}
```

### 3. MULLIGAN State

Each player independently decides whether to keep their opening hand or take a mulligan. MTGNP uses the London Mulligan rule: a player who mulligans draws a new hand of seven cards, then puts a number of cards on the bottom of their library equal to the number of times they have mulliganed.

In this example, Player 1 keeps their opening hand immediately, while Player 2 takes one mulligan before keeping — and must therefore bottom exactly 1 card.

#### Step 7 - Player 1 keeps their opening hand

Player 1 is satisfied with their hand. seq\_num echoes the GAME\_STATE\_UPDATE from Step 5 (seq\_num 3). No cards are bottomed since Player 1 has not mulliganed.<sup>1</sup>

#### C -> S (Player 1)

```
{
  "type": "MULLIGAN_CHOICE", "seq_num": 3, "keep": true, "cards_to_bottom": []
}
```

#### Step 8 - Player 2 takes a mulligan

Player 2 is not happy with their opening hand. seq\_num echoes the GAME\_STATE\_UPDATE from Step 6 (seq\_num 3). cards\_to\_bottom is empty when keep is false.

#### C -> S (Player 2)

```
{
  "type": "MULLIGAN_CHOICE", "seq_num": 3, "keep": false, "cards_to_bottom": []
}
```

#### Step 9 - Server redraws 7 cards for Player 2

---

<sup>1</sup>A player who has never mulliganed keeps with an empty cards\_to\_bottom array, since N = 0.

The server draws a fresh 7-card hand for Player 2 and sends a new personalized GAME\_STATE\_UPDATE. seq\_num advances to 4. Player 1 does not receive a new update.<sup>2</sup>

**S -> P2**

```
{
  "type": "GAME_STATE_UPDATE",
  "seq_num": 4,
  "state": {
    "turn": 0, "phase": "MULLIGAN", "active_player": "player_1",
    "life_totals": { "player_1": 20, "player_2": 20 },
    "hand":
    ["counterspell_001","island_001","swamp_001","island_002","gray_merchant_001","swamp_002","counterspell_002"],
    "hand_counts": { "player_1": 7 }, "library_counts": { "player_1": 1,
    "player_2": 1 },
    "battlefield": { "player_1": [], "player_2": [] },
    "graveyard": { "player_1": [], "player_2": [] }, "stack": []
  }
}
```

### Step 10 - Player 2 keeps after mulligan, bottoms 1 card

Player 2 keeps the new hand. Because they mulliganed once, cards\_to\_bottom MUST contain exactly 1 card ID. seq\_num echoes the redraw GAME\_STATE\_UPDATE from Step 9 (seq\_num 4).<sup>3</sup>

**C -> S (Player 2)**

```
{
  "type": "MULLIGAN_CHOICE", "seq_num": 4, "keep": true, "cards_to_bottom":
  ["counterspell_002"]
}
```

### Step 11 - Both players have kept; server transitions to IN\_GAME

Both players have now sent MULLIGAN\_CHOICE with keep: true. The server transitions to IN\_GAME and begins Player 1's first turn, broadcasting a PHASE\_TRANSITION to all players.<sup>4</sup>

seq\_num 5 on the PHASE\_TRANSITION continues the server counter from the last GAME\_STATE\_UPDATE sent to Player 2 (seq\_num 4).<sup>5</sup>

**S -> ALL**

```
{
  "type": "PHASE_TRANSITION", "seq_num": 5,
  "from_phase": "MULLIGAN", "to_phase": "UNTAP",
  "active_player": "player_1", "turn": 1
}
```

---

<sup>2</sup>Only the mulliganing player receives a new GAME\_STATE\_UPDATE after a redraw. The other player receives no PDU until both have kept and the server broadcasts PHASE\_TRANSITION.

<sup>3</sup>When keep is false, cards\_to\_bottom MUST be empty. When keep is true, cards\_to\_bottom MUST contain exactly N card IDs where N equals the number of mulligans taken. The server rejects a mismatch with ERROR code ILLEGAL\_ACTION.

<sup>4</sup>Players decide independently — each player's MULLIGAN\_CHOICE is processed separately. Player 1's keep does not block or affect Player 2's mulligan decision.

<sup>5</sup>seq\_num on PHASE\_TRANSITION continues the server counter from the last GAME\_STATE\_UPDATE sent to either player — here seq\_num 5 follows seq\_num 4, the redraw sent to Player 2.



```
}
```

---

## 4. IN\_GAME State — Turn 1 (Player 1)

---

Player 1 is the Active Player (AP). Player 2 is the Non-Active Player (NAP). The turn follows the full phase sequence: Untap -> Upkeep -> Draw -> Precombat Main -> Combat -> Postcombat Main -> End Step -> Cleanup.

### Step 12 - Untap Step (automatic, no priority)

The server untaps all of Player 1's permanents and resets `land_played_this_turn` to false. No priority is granted. The server immediately advances to Upkeep.<sup>6</sup>

**S -> ALL**

```
{
  "type": "PHASE_TRANSITION", "seq_num": 6,
  "from_phase": "MULLIGAN", "to_phase": "UNTAP",
  "active_player": "player_1", "turn": 1
}
```

**S -> ALL (untap broadcast)**

```
{
  "type": "GAME_STATE_UPDATE", "seq_num": 7,
  "state": {
    "turn": 1, "phase": "UNTAP", "active_player": "player_1",
    "life_totals": { "player_1": 20, "player_2": 20 }, "land_played": false,
    "battlefield": { "player_1": [], "player_2": [] },
    "hand_counts": { "player_1": 7, "player_2": 6 },
    "library_counts": { "player_1": 1, "player_2": 1 }, "stack": []
  }
}
```

**S -> ALL (advance to Upkeep)**

```
{
  "type": "PHASE_TRANSITION", "seq_num": 8,
  "from_phase": "UNTAP", "to_phase": "UPKEEP",
  "active_player": "player_1", "turn": 1
}
```

### Step 13 - Upkeep Step (both players pass, no actions)

---

<sup>6</sup>The Untap Step has no priority window. The server performs all untap actions automatically and advances to Upkeep without waiting for any client PDU.

The server opens a priority window. Player 1 holds priority first. Both players pass with an empty stack, so the server advances to Draw.<sup>7</sup>

#### **S -> P1**

```
{
  "type": "PRIORITY_GRANT", "player_id": "player_1", "seq_num": 8, "time_limit_ms":
60000
}
```

#### **C -> S (Player 1 passes)**

```
{
  "type": "PRIORITY_PASS", "seq_num": 8
}
```

#### **S -> P2**

```
{
  "type": "PRIORITY_GRANT", "player_id": "player_2", "seq_num": 9, "time_limit_ms":
60000
}
```

#### **C -> S (Player 2 passes)**

```
{
  "type": "PRIORITY_PASS", "seq_num": 9
}
```

#### **S -> ALL (both passed, empty stack — advance to Draw)**

```
{
  "type": "PHASE_TRANSITION", "seq_num": 10,
  "from_phase": "UPKEEP", "to_phase": "DRAW",
  "active_player": "player_1", "turn": 1
}
```

### **Step 14 - Draw Step (Player 1 draws, both pass)**

The server draws one card for Player 1 and sends a personalized GAME\_STATE\_UPDATE. A priority window opens; both players pass and the server advances to Precombat Main.<sup>8</sup>

#### **S -> P1 (shock\_002 added to hand)**

---

<sup>7</sup>Every priority window follows the same pattern: PRIORITY\_GRANT to AP, PRIORITY\_PASS from AP, PRIORITY\_GRANT to NAP, PRIORITY\_PASS from NAP — then PHASE\_TRANSITION if the stack is empty. Only the Precombat Main spell-casting window is shown in full detail; other windows are summarised for brevity.

<sup>8</sup>Per the RFC, on the very first turn the first player does NOT draw a card during the Draw Step. This example shows a representative turn with a draw for clarity; a strict Turn 1 implementation would skip the card draw and open the priority window on an unchanged hand.

```
{
  "type": "GAME_STATE_UPDATE", "seq_num": 11,
  "state": {
    "turn": 1, "phase": "DRAW", "active_player": "player_1",
    "life_totals": { "player_1": 20, "player_2": 20 },
    "hand":
["lightning_bolt_001","shock_001","mountain_001","mountain_002","goblin_guide_001",
"lightning_bolt_002","mountain_003","shock_002"],
    "hand_counts": { "player_2": 6 },
    "library_counts": { "player_1": 0, "player_2": 1 }, "stack": []
  }
}
```

Priority exchange follows (Player 1 passes, Player 2 passes, empty stack). Server advances.

**S -> ALL (advance to Precombat Main)**

```
{
  "type": "PHASE_TRANSITION", "seq_num": 14,
  "from_phase": "DRAW", "to_phase": "PRECOMBAT_MAIN",
  "active_player": "player_1", "turn": 1
}
```

## Step 15 - Precombat Main Phase: Play a Land

Player 1 plays mountain\_003 as their land for the turn. Playing a land does not use the stack and does not require priority. The server updates state and re-issues PRIORITY\_GRANT to Player 1.<sup>9</sup>

**C -> S (Player 1 plays land)**

```
{
  "type": "PLAY_LAND", "seq_num": 14, "card_id": "mountain_003"
}
```

**S -> ALL (land enters battlefield)**

```
{
  "type": "GAME_STATE_UPDATE", "seq_num": 15,
  "state": {
    "turn": 1, "phase": "PRECOMBAT_MAIN", "active_player": "player_1",
    "life_totals": { "player_1": 20, "player_2": 20 }, "land_played": true,
    "battlefield": { "player_1": [{ "id": "mountain_003", "tapped": false }],
"player_2": [] },
    "hand_counts": { "player_1": 7, "player_2": 6 }, "stack": []
  }
}
```

**S -> P1 (re-issue priority after land)**

```
{
```

---

<sup>9</sup>PLAY\_LAND bypasses the stack entirely. The server deducts the land from the hand, places it on the battlefield, sets land\_played to true, and re-issues PRIORITY\_GRANT to the Active Player.

```
{
  "type": "PRIORITY_GRANT", "player_id": "player_1", "seq_num": 15,
  "time_limit_ms": 60000
}
```

## Step 16 - Precombat Main Phase: Cast Goblin Guide

Player 1 casts goblin\_guide\_001, paying 1 Red mana. Both players pass priority consecutively, the spell resolves, and Goblin Guide enters the battlefield.

### C -> S (Player 1 casts Goblin Guide)

```
{
  "type": "CAST_SPELL", "seq_num": 15,
  "card_id": "goblin_guide_001", "targets": [], "mana_payment": { "R": 1 }
}
```

### S -> ALL (spell pushed to stack)

```
{
  "type": "STACK_PUSH", "seq_num": 16,
  "stack_item_id": "stk_01", "item_type": "SPELL",
  "source": "goblin_guide_001", "targets": [], "controller": "player_1"
}
```

### S -> P1 (AP retains priority)

```
{
  "type": "PRIORITY_GRANT", "player_id": "player_1", "seq_num": 16,
  "time_limit_ms": 60000
}
```

### C -> S (Player 1 passes)

```
{
  "type": "PRIORITY_PASS", "seq_num": 16
}
```

### S -> P2

```
{
  "type": "PRIORITY_GRANT", "player_id": "player_2", "seq_num": 17,
  "time_limit_ms": 60000
}
```

### C -> S (Player 2 passes — no response)

```
{
  "type": "PRIORITY_PASS", "seq_num": 17
}
```

Both players passed consecutively with a non-empty stack — server resolves the top item.

**S -> ALL (Goblin Guide resolves, enters battlefield)**

```
{
  "type": "STACK_RESOLVE", "seq_num": 18,
  "stack_item_id": "stk_01", "result": "RESOLVED",
  "state_changes": [{ "type": "PERMANENT_ENTERS", "card_id": "goblin_guide_001",
    "controller": "player_1", "tapped": false }]
}
```

**S -> ALL (updated battlefield)**

```
{
  "type": "GAME_STATE_UPDATE", "seq_num": 19,
  "state": {
    "turn": 1, "phase": "PRECOMBAT_MAIN", "active_player": "player_1",
    "life_totals": { "player_1": 20, "player_2": 20 },
    "battlefield": { "player_1": [
      { "id": "mountain_003", "tapped": false },
      { "id": "goblin_guide_001", "tapped": false, "summoning_sickness": true }
    ], "player_2": [] },
    "hand_counts": { "player_1": 6, "player_2": 6 }, "stack": []
  }
}
```

Both players pass priority again with an empty stack. Server advances to Combat.

**S -> ALL (advance to Combat)**

```
{
  "type": "PHASE_TRANSITION", "seq_num": 22,
  "from_phase": "PRECOMBAT_MAIN", "to_phase": "BEGIN_COMBAT",
  "active_player": "player_1", "turn": 1
}
```

## Step 17 - Begin Combat Step (both pass)

Priority window opens at Begin Combat. Both players pass with an empty stack.

**S -> ALL (advance to Declare Attackers)**

```
{
  "type": "PHASE_TRANSITION", "seq_num": 25,
  "from_phase": "BEGIN_COMBAT", "to_phase": "DECLARE_ATTACKERS",
  "active_player": "player_1", "turn": 1
}
```

## Step 18 - Declare Attackers Step

Goblin Guide entered the battlefield this turn, so it has summoning sickness and MUST NOT attack. Player 1 has no other attackers, so they declare no attackers. The server advances past combat.<sup>10</sup>

---

<sup>10</sup>A creature has summoning sickness the turn it enters the battlefield. It MUST NOT be declared as an attacker and MUST NOT activate tap abilities until the controller's next Untap Step.

### **S -> P1 (priority to declare attackers)**

```
{
  "type": "PRIORITY_GRANT", "player_id": "player_1", "seq_num": 25,
  "time_limit_ms": 60000
}
```

### **C -> S (Player 1 declares no attackers)**

```
{
  "type": "DECLARE_ATTACKERS", "seq_num": 25, "attackers": []
}
```

With no attackers declared, the server skips Declare Blockers, Assign Damage Order, and Combat Damage, advancing directly to End of Combat.

### **S -> ALL (skip to End of Combat)**

```
{
  "type": "PHASE_TRANSITION", "seq_num": 26,
  "from_phase": "DECLARE_ATTACKERS", "to_phase": "END_OF_COMBAT",
  "active_player": "player_1", "turn": 1
}
```

## **Step 19 - Postcombat Main Phase (both pass)**

Priority window opens. Player 1 takes no further actions. Both pass with empty stack.

### **S -> ALL (advance to Postcombat Main)**

```
{
  "type": "PHASE_TRANSITION", "seq_num": 29,
  "from_phase": "END_OF_COMBAT", "to_phase": "POSTCOMBAT_MAIN",
  "active_player": "player_1", "turn": 1
}
```

Both players pass priority. Server advances to End Step.

### **S -> ALL**

```
{
  "type": "PHASE_TRANSITION", "seq_num": 32,
  "from_phase": "POSTCOMBAT_MAIN", "to_phase": "END_STEP",
  "active_player": "player_1", "turn": 1
}
```

## **Step 20 - End Step (both pass)**

A final priority window opens. Both players pass with an empty stack. Server advances to Cleanup.

### **S -> ALL**

```
{
```

```

    "type": "PHASE_TRANSITION", "seq_num": 35,
    "from_phase": "END_STEP", "to_phase": "CLEANUP",
    "active_player": "player_1", "turn": 1
  }

```

## Step 21 - Cleanup Step

Player 1 has 6 cards in hand (under the 7-card limit), so no discard is needed. The server clears all damage from creatures and removes until-end-of-turn effects, then broadcasts a final GAME\_STATE\_UPDATE. Summoning sickness is cleared from Goblin Guide. No priority is granted. The server increments the turn counter, switches the Active Player to Player 2, and begins Turn 2's Untap Step.

**S -> ALL (damage cleared, summoning sickness removed)**

```

{
  "type": "GAME_STATE_UPDATE", "seq_num": 36,
  "state": {
    "turn": 1, "phase": "CLEANUP", "active_player": "player_1",
    "life_totals": { "player_1": 20, "player_2": 20 },
    "battlefield": { "player_1": [
      { "id": "mountain_003", "tapped": false },
      { "id": "goblin_guide_001", "tapped": false, "summoning_sickness": false }
    ], "player_2": [] },
    "hand_counts": { "player_1": 6, "player_2": 6 }, "stack": []
  }
}

```

**S -> ALL (Turn 2 begins — Player 2 is now Active Player)**

```

{
  "type": "PHASE_TRANSITION", "seq_num": 37,
  "from_phase": "CLEANUP", "to_phase": "UNTAP",
  "active_player": "player_2", "turn": 2
}

```

## 5. IN\_GAME State — Turn 2 (Player 2)

Player 2 is now the Active Player (AP). Player 1 is the Non-Active Player (NAP). Player 2 enters with 6 cards in hand, no permanents on the battlefield. Player 1 has mountain\_003 and goblin\_guide\_001 in play. The Turn 2 UNTAP PHASE\_TRANSITION was already broadcast at seq\_num 37.

## Step 22 - Untap Step (automatic, no priority)

Player 2 has no permanents to untap. The server resets land\_played\_this\_turn for Player 2 and immediately advances to Upkeep.

**S -> ALL (state after untap)**

```

{
  "type": "GAME_STATE_UPDATE", "seq_num": 38,
  "state": {

```

```

    "turn": 2, "phase": "UNTAP", "active_player": "player_2",
    "life_totals": { "player_1": 20, "player_2": 20 }, "land_played": false,
    "battlefield": { "player_1": [
      { "id": "mountain_003", "tapped": false },
      { "id": "goblin_guide_001", "tapped": false }
    ], "player_2": [] },
    "hand_counts": { "player_1": 6, "player_2": 6 },
    "library_counts": { "player_1": 0, "player_2": 1 }, "stack": []
  }
}

```

### **S -> ALL (advance to Upkeep)**

```

{
  "type": "PHASE_TRANSITION", "seq_num": 39,
  "from_phase": "UNTAP", "to_phase": "UPKEEP",
  "active_player": "player_2", "turn": 2
}

```

## **Step 23 - Upkeep Step (both pass)**

Priority opens with Player 2 holding priority first. Neither player takes action. Both pass consecutively with an empty stack.

### **S -> P2**

```

{
  "type": "PRIORITY_GRANT", "player_id": "player_2", "seq_num": 39,
  "time_limit_ms": 60000
}

```

### **C -> S (Player 2 passes)**

```

{
  "type": "PRIORITY_PASS", "seq_num": 39
}

```

### **S -> P1**

```

{
  "type": "PRIORITY_GRANT", "player_id": "player_1", "seq_num": 40,
  "time_limit_ms": 60000
}

```

### **C -> S (Player 1 passes)**

```

{
  "type": "PRIORITY_PASS", "seq_num": 40
}

```

### **S -> ALL (advance to Draw)**



```
{
  "type": "PHASE_TRANSITION", "seq_num": 41,
  "from_phase": "UPKEEP", "to_phase": "DRAW",
  "active_player": "player_2", "turn": 2
}
```

## Step 24 - Draw Step (Player 2 draws island\_002)

The server draws one card for Player 2. Player 2 now has 7 cards in hand. A priority window opens; both players pass and the server advances to Precombat Main.<sup>11</sup>

### S -> P2 (personalized update with new card)

```
{
  "type": "GAME_STATE_UPDATE", "seq_num": 42,
  "state": {
    "turn": 2, "phase": "DRAW", "active_player": "player_2",
    "life_totals": { "player_1": 20, "player_2": 20 },
    "hand":
    ["counterspell_001", "gray_merchant_001", "island_001", "swamp_001", "gray_merchant_002",
    "swamp_002", "island_002"],
    "hand_counts": { "player_1": 6 },
    "library_counts": { "player_1": 0, "player_2": 0 }, "stack": []
  }
}
```

Priority exchange: Player 2 passes, Player 1 passes, empty stack. Server advances.

### S -> ALL (advance to Precombat Main)

```
{
  "type": "PHASE_TRANSITION", "seq_num": 45,
  "from_phase": "DRAW", "to_phase": "PRECOMBAT_MAIN",
  "active_player": "player_2", "turn": 2
}
```

## Step 25 - Precombat Main Phase: Play a Land

Player 2 plays swamp\_001. The server places it on the battlefield, sets land\_played to true, and re-issues PRIORITY\_GRANT to Player 2.

### C -> S (Player 2 plays land)

```
{
  "type": "PLAY_LAND", "seq_num": 45, "card_id": "swamp_001"
}
```

### S -> ALL (swamp\_001 enters battlefield)

```
{
```

---

<sup>11</sup>Player 2's library reaches 0 after this draw. An 8-card deck minus 7 drawn at setup minus 1 bottomed during mulligan leaves 0 cards. Any draw attempt on Turn 3 or later triggers the DECK\_EMPTY win condition.

```

"type": "GAME_STATE_UPDATE", "seq_num": 46,
"state": {
  "turn": 2, "phase": "PRECOMBAT_MAIN", "active_player": "player_2",
  "life_totals": { "player_1": 20, "player_2": 20 }, "land_played": true,
  "battlefield": { "player_1": [
    { "id": "mountain_003", "tapped": false },
    { "id": "goblin_guide_001", "tapped": false }
  ], "player_2": [{ "id": "swamp_001", "tapped": false }] },
  "hand_counts": { "player_1": 6, "player_2": 6 }, "stack": []
}
}

```

### S -> P2 (re-issue priority after land)

```

{
  "type": "PRIORITY_GRANT", "player_id": "player_2", "seq_num": 46,
  "time_limit_ms": 60000
}

```

## Step 26 - Precombat Main Phase: Player 1 casts Lightning Bolt

Player 2 passes priority. Player 1 (NAP) receives priority and casts lightning\_bolt\_001 targeting Player 2, tapping mountain\_003 to pay 1 Red mana. Player 2 holds counterspell\_001 but only has swamp\_001 available — they cannot pay the UU cost. Player 2 passes. Lightning Bolt resolves, dealing 3 damage to Player 2.<sup>12</sup>

### C -> S (Player 2 passes)

```

{
  "type": "PRIORITY_PASS", "seq_num": 46
}

```

### S -> P1 (NAP receives priority)

```

{
  "type": "PRIORITY_GRANT", "player_id": "player_1", "seq_num": 47,
  "time_limit_ms": 60000
}

```

### C -> S (Player 1 casts Lightning Bolt targeting Player 2)

```

{
  "type": "CAST_SPELL", "seq_num": 47,
  "card_id": "lightning_bolt_001", "targets": ["player_2"],
  "mana_payment": { "R": 1 }
}

```

### S -> ALL (Lightning Bolt pushed to stack)

---

<sup>12</sup>Player 1 receiving priority during Player 2's Precombat Main Phase is legal. When the Active Player passes priority, the Non-Active Player receives it and may cast instants or activate abilities at instant speed.

```
{
  "type": "STACK_PUSH", "seq_num": 48,
  "stack_item_id": "stk_02", "item_type": "SPELL",
  "source": "lightning_bolt_001", "targets": ["player_2"], "controller": "player_1"
}
```

#### **S -> P1 (AP retains priority after casting)**

```
{
  "type": "PRIORITY_GRANT", "player_id": "player_1", "seq_num": 48,
  "time_limit_ms": 60000
}
```

#### **C -> S (Player 1 passes)**

```
{
  "type": "PRIORITY_PASS", "seq_num": 48
}
```

#### **S -> P2**

```
{
  "type": "PRIORITY_GRANT", "player_id": "player_2", "seq_num": 49,
  "time_limit_ms": 60000
}
```

#### **C -> S (Player 2 passes — cannot pay UU for Counterspell)**

```
{
  "type": "PRIORITY_PASS", "seq_num": 49
}
```

Both players passed consecutively with a non-empty stack. Server resolves the top item.<sup>13</sup>

#### **S -> ALL (Lightning Bolt resolves — 3 damage to Player 2)**

```
{
  "type": "STACK_RESOLVE", "seq_num": 50,
  "stack_item_id": "stk_02", "result": "RESOLVED",
  "state_changes": [{ "type": "DAMAGE", "target": "player_2", "amount": 3 }]
}
```

#### **S -> ALL (updated life totals)**

```
{
  "type": "GAME_STATE_UPDATE", "seq_num": 51,
  "state": {
    "turn": 2, "phase": "PRECOMBAT_MAIN", "active_player": "player_2",

```

---

<sup>13</sup>Counterspell costs UU. Player 2's only mana source is swamp\_001, which produces Black mana. Had Player 2 attempted the cast, the server would have returned ERROR code INSUFFICIENT\_MANA.

```

    "life_totals": { "player_1": 20, "player_2": 17 }, "land_played": true,
    "battlefield": { "player_1": [
      { "id": "mountain_003", "tapped": true },
      { "id": "goblin_guide_001", "tapped": false }
    ], "player_2": [{ "id": "swamp_001", "tapped": false }] },
    "hand_counts": { "player_1": 5, "player_2": 6 }, "stack": []
  }
}

```

Priority re-opens. Both players pass with an empty stack. Server advances to Combat.

**S -> ALL (advance to Combat)**

```

{
  "type": "PHASE_TRANSITION", "seq_num": 54,
  "from_phase": "PRECOMBAT_MAIN", "to_phase": "BEGIN_COMBAT",
  "active_player": "player_2", "turn": 2
}

```

## Step 27 - Combat Phase (no attackers)

Player 2 has no creatures on the battlefield and declares no attackers. Both players pass at Begin Combat. The server skips to End of Combat.

**S -> ALL (advance to Declare Attackers)**

```

{
  "type": "PHASE_TRANSITION", "seq_num": 57,
  "from_phase": "BEGIN_COMBAT", "to_phase": "DECLARE_ATTACKERS",
  "active_player": "player_2", "turn": 2
}

```

**S -> P2**

```

{
  "type": "PRIORITY_GRANT", "player_id": "player_2", "seq_num": 57,
  "time_limit_ms": 60000
}

```

**C -> S (Player 2 declares no attackers)**

```

{
  "type": "DECLARE_ATTACKERS", "seq_num": 57, "attackers": []
}

```

**S -> ALL (skip to End of Combat)**

```

{
  "type": "PHASE_TRANSITION", "seq_num": 58,
  "from_phase": "DECLARE_ATTACKERS", "to_phase": "END_OF_COMBAT",
  "active_player": "player_2", "turn": 2
}

```

## Step 28 - Postcombat Main Phase (both pass)

Priority window opens. Player 2 has no further actions. Both pass with an empty stack.

**S -> ALL (advance to Postcombat Main)**

```
{
  "type": "PHASE_TRANSITION", "seq_num": 61,
  "from_phase": "END_OF_COMBAT", "to_phase": "POSTCOMBAT_MAIN",
  "active_player": "player_2", "turn": 2
}
```

**S -> ALL (advance to End Step)**

```
{
  "type": "PHASE_TRANSITION", "seq_num": 64,
  "from_phase": "POSTCOMBAT_MAIN", "to_phase": "END_STEP",
  "active_player": "player_2", "turn": 2
}
```

## Step 29 - End Step (both pass)

Final priority window of the turn. Both players pass with an empty stack.

**S -> ALL (advance to Cleanup)**

```
{
  "type": "PHASE_TRANSITION", "seq_num": 67,
  "from_phase": "END_STEP", "to_phase": "CLEANUP",
  "active_player": "player_2", "turn": 2
}
```

## Step 30 - Cleanup Step

Player 2 has 6 cards in hand (under the 7-card limit — drew 1, played swamp\_001), so no discard is needed. The server clears damage markers from all creatures. mountain\_003 untaps at the start of Player 1's next turn, not here. The server increments the turn counter, switches the Active Player to Player 1, and begins Turn 3's Untap Step.<sup>14</sup>

**S -> ALL (damage cleared)**

```
{
  "type": "GAME_STATE_UPDATE", "seq_num": 68,
  "state": {
    "turn": 2, "phase": "CLEANUP", "active_player": "player_2",
    "life_totals": { "player_1": 20, "player_2": 17 },
    "battlefield": { "player_1": [
      { "id": "mountain_003", "tapped": true },
      { "id": "goblin_guide_001", "tapped": false }
    ], "player_2": [{ "id": "swamp_001", "tapped": false }] },
    "hand_counts": { "player_1": 5, "player_2": 6 }, "stack": []
  }
}
```

---

<sup>14</sup>mountain\_003 remains tapped through Cleanup — lands are not untapped during the owner's Cleanup Step. They untap at the start of the owner's next Untap Step.

**S -> ALL (Turn 3 begins — Player 1 is Active Player again)**

```
{
  "type": "PHASE_TRANSITION", "seq_num": 69,
  "from_phase": "CLEANUP", "to_phase": "UNTAP",
  "active_player": "player_1", "turn": 3
}
```

## 6. GAME\_OVER State

NOTE: Several turns have elapsed between Turn 2 and the exchange shown below. Over the course of those turns, Goblin Guide attacked repeatedly, and Player 1 used additional burn spells to reduce Player 2's life total. Player 2 managed to deploy Gray Merchant of Asphodel, draining Player 1 for some life, but was never able to stabilize the board. By Turn 7, the game state entering Player 1's Precombat Main Phase is as follows:

```
Life totals:  Player 1 = 14,  Player 2 = 3
Battlefield:
  Player 1: mountain_001, mountain_002, mountain_003 (all untapped),
            goblin_guide_001 (untapped)
  Player 2: swamp_001, island_001 (both untapped)
Player 1 hand: lightning_bolt_003, shock_002
Player 2 hand: counterspell_001
seq_num at start of this exchange: 118
```

### Step 31 - Player 1 casts Lightning Bolt targeting Player 2 (lethal)

Player 1 casts lightning\_bolt\_003, targeting Player 2 who is at 3 life. The Bolt deals 3 damage — exactly enough to reduce Player 2's life total to 0.

**S -> P1 (priority granted in Precombat Main)**

```
{
  "type": "PRIORITY_GRANT", "player_id": "player_1", "seq_num": 118,
  "time_limit_ms": 60000
}
```

**C -> S (Player 1 casts Lightning Bolt)**

```
{
  "type": "CAST_SPELL", "seq_num": 118,
  "card_id": "lightning_bolt_003", "targets": ["player_2"],
  "mana_payment": { "R": 1 }
}
```

**S -> ALL (Lightning Bolt pushed to stack)**

```
{
  "type": "STACK_PUSH", "seq_num": 119,
```

```
{
  "stack_item_id": "stk_09", "item_type": "SPELL",
  "source": "lightning_bolt_003", "targets": ["player_2"], "controller": "player_1"
}
```

#### **S -> P1 (AP retains priority)**

```
{
  "type": "PRIORITY_GRANT", "player_id": "player_1", "seq_num": 119,
  "time_limit_ms": 60000
}
```

#### **C -> S (Player 1 passes)**

```
{
  "type": "PRIORITY_PASS", "seq_num": 119
}
```

#### **S -> P2**

```
{
  "type": "PRIORITY_GRANT", "player_id": "player_2", "seq_num": 120,
  "time_limit_ms": 60000
}
```

#### **C -> S (Player 2 passes — counterspell\_001 requires UU, only BU available)**

```
{
  "type": "PRIORITY_PASS", "seq_num": 120
}
```

Both players passed consecutively. Server resolves the top item.

#### **S -> ALL (Lightning Bolt resolves — 3 damage to Player 2)**

```
{
  "type": "STACK_RESOLVE", "seq_num": 121,
  "stack_item_id": "stk_09", "result": "RESOLVED",
  "state_changes": [{ "type": "DAMAGE", "target": "player_2", "amount": 3 }]
}
```

## **Step 32 - Server detects win condition and broadcasts GAME\_OVER**

Player 2's life total has reached 0. The server immediately detects the LIFE\_ZERO win condition, skips any further priority windows, and broadcasts GAME\_OVER to all connected players. Player 1 is declared the winner.<sup>15</sup>

#### **S -> ALL**

---

<sup>15</sup>LIFE\_ZERO is detected immediately after STACK\_RESOLVE applies damage — no further priority windows are granted before GAME\_OVER is broadcast.

```
{
  "type":      "GAME_OVER",
  "seq_num":   122,
  "winner_id": "player_1",
  "loser_id":  "player_2",
  "reason":    "LIFE_ZERO"
}
```

The reason field identifies how the game ended.<sup>16</sup>

winner\_id is always set to the non-offending or surviving player.<sup>17</sup>

### Step 33 - Server transitions back to LOBBY

Immediately after broadcasting GAME\_OVER, the server transitions back to LOBBY state. The existing TCP connections are retained. Both players must send a fresh PLAYER\_READY PDU to begin a new game. The server does not broadcast a PHASE\_TRANSITION for this — the GAME\_OVER PDU itself signals the return to LOBBY.<sup>18</sup>

---

<sup>16</sup>Valid reason values: LIFE\_ZERO (life total reaches 0), DECK\_EMPTY (draw from empty library), CONCEDE (player sends CONCEDE PDU), DISCONNECT (connection lost, reconnect timer expired).

<sup>17</sup>winner\_id is the non-offending or surviving player in all cases.

<sup>18</sup>TCP connections are preserved across GAME\_OVER. Both players can start a new game immediately by sending PLAYER\_READY on the same connection — no reconnection required.